CSE493:LINUX SYSTEM ADMINISTRATION

L:3 T:0 P:2 Credits:4

Course Outcomes: Through this course students should be able to

CO1 :: understand fundamental Linux concepts and help documentation tools to effectively navigate and manage the Linux environment.

CO2 :: apply Linux command-line tools to manage files, users, permissions, processes, and services securely and efficiently

CO3 :: analyze logs, processes, networking, and package management to diagnose system issues and performance

CO4 :: establish networking, SSH security, scheduled tasks, and services for reliable remote and automated operations

CO5 :: apply secure storage techniques using logical volumes, ACLs, SELinux, compression, deduplication, and NFS.

CO6 :: demonstrate Linux system installation, boot management, and troubleshooting using automation tools.

Unit I

Accessing the command line : Accessing command line using local console, Accessing command line using desktop, Executing commands using bash shell

Managing Files From the Command Line : The Linux File System Hierarchy, Locating Files by Name, Managing Files Using Command-Line Tools, Matching File Names Using Path Name Expansion, Managing Files with Shell Expansion

Getting Help in Red Hat Enterprise Linux : Reading Documentation Using man Command, Reading Documentation Using pinfo Command, Reading Documentation in /usr/share/doc, Getting Help From Red Hat

Creating, Viewing, and Editing Text Files : Redirecting Output to a File or Program, Editing Text Files from the Shell Prompt, Editing Text Files with a Graphical Editor

Managing Local Linux Users and Groups : Users and Groups, Gaining Superuser Access, Managing Local User Accounts, Managing Local Group Accounts, Managing User Passwords

Unit II

Controlling Access to Files with Linux File System Permissions : Linux File System Permissions, Managing File System Permissions from the Command Line, Managing Default Permissions and File Access

Monitoring and Managing Linux Processes : Processes, Controlling Jobs, Killing Processes, Monitoring Process Activity

Controlling Services and Daemons : Identifying Automatically Started System Processes, Controlling System Services

Configuring and Securing OpenSSH Service : Accessing the Remote Command Line with SSH, Configuring SSH Key-based Authentication, Customizing SSH Service Configuration

Analyzing and Storing Logs : System Log Architecture, Reviewing Sys log Files, Reviewing systemd Journal Entries, Preserving the systemd Journal, Maintaining Accurate Time

Unit III

Managing Red Hat Enterprise Linux Networking : Networking Concepts, Validating Network Configuration, Configuring Networking with nmcli, Editing Network Configuration Files, Configuring Host Names and Name Resolution

Archiving and Copying Files Between Systems : Managing Compressed tar Archives, Copying Files Between Systems Securely, Synchronizing Files Between Systems Securely

Installing and Updating Software Packages : Managing Software Updates with yum, Enabling yum Software Repositories, Examining RPM Package Files, Attaching Systems to Subscriptions for Software Updates, RPM Software Packages and Yum

Accessing Linux File Systems : Identifying File Systems and Devices, Mounting and Unmounting File Systems, Making Links Between Files Locating Files on the System

Using Virtualized Systems : Managing a Local Virtualization Host, Installing a New Virtual Machine

Unit IV

Improving Command-line productivity : Writing Simple Bash Scripts, Running commands More Efficiently Using Loops, Matching Text in Command Output with Regular Expressions

Unit IV

Scheduling Future Tasks : Scheduling a Deferred User Job, Scheduling Recurring System Jobs, Managing Temporary Files

Tuning System Performance : Adjusting Tuning Profiles, Influencing Process Scheduling

Controlling Access to Files with ACLs : Interpreting File ACLs, Securing Files with ACLs

Unit V

Managing SELinux Security : Changing the SELinux Enforcement Mode, Controlling SELinux File Contexts, Controlling SELinux File Contexts, Adjusting SELinux Policy with Booleans, Investing and Resolving SELinux Issues

Managing Basic Storage and Logical Volumes : Adding Partitions, File Systems, and Persistent Mounts, Managing Swap Space, Creating Logical Volumes, Extended Logical Volumes

Implementing Advanced Storage Features : Managing Layered Storage with Stratis, Compressing and Deduplicating Storage with VDO

Unit VI

Accessing Network-Attached Storage : Mounting Network-Attached Storage with NFS, Automounting Network-Attached Storage

Controlling the Boot Process : Selecting the Boot Target, Resetting the Root Password, Repairing File System Issues at Boot

Managing Network Security : Managing Server Firewalls, Controlling SeLinux Port Labeling

Installing Red Hat Enterprise Linux : Installing RedHat EnterpriseLinux, Automating Installation with Kickstart, Installing and Configuring Virtual Machines

List of Practicals / Experiments:

List of Practical's

- Executing commands using bash shell
- Locating Files by Name
- Editing Text Files from the Shell Prompt
- Managing Local User Accounts and Group Accounts
- Managing File System Permissions from the Command Line
- Killing and Monitoring Process Activity
- Configuring SSH Key-based Authentication
- Validating Network Configuration and Configuring Networking with nmcli
- Installing a New Virtual Machine
- Matching Text in Command Output with Regular Expressions
- Interpreting and Securing Files with ACLs
- Compressing and Deduplicating Storage with VDO
- Mounting Network-Attached Storage with NFS
- Managing Server Firewalls and Controlling SeLinux Port Labeling

Text Books:

1. RHCSA/RHCE RED HAT LINUX CERTIFICATION STUDY GUIDE, SEVENTH EDITION (EXAMS EX200 & EX300) (RHCSA/RHCE RED HAT ENTERPRISE LINUX CERTIFICATION STUDY GUIDE) by MICHAEL JANG, ALESSANDRO ORSARIA, Tata McGraw Hill, India

References:

1. RHCSA & RHCE RED HAT ENTERPRISE LINUX 7: TRAINING AND EXAM PREPARATION GUIDE (EX200 AND EX300) by ASGHAR GHORI, LIGHTNING SOURCE INCORPORATED

Linux System Administration

Exam Quick Revision & Practical Command Guide

CSE 493 · Units I–VI

Prepared from the course syllabus, Linux presentations, practical questions, shell notes, networking/storage examples, and revision files in **linuxppt1**.

How to use tonight: Read the command tables once, practise the 15 Unit I–II scenarios, then revise the bold “must remember” points. Never run destructive storage commands on a real machine without confirming the device.

Linux commands may vary slightly between RHEL versions. Use `man command` and `command --help` when in doubt.

1. Exam strategy and command pattern

Scenario → command → verify. For almost every practical, first inspect the current state, make one change, then verify with a read-only command. Use `sudo` only when required.

Need	Useful commands
Current location and files	`pwd`, `ls -la`, `cd /path`
Help	`man cmd`, `cmd --help`, `info cmd`, `apropos keyword`
Verify identity	`whoami`, `id`, `hostname`
Read safely	`cat`, `less`, `head`, `tail`
Find and filter	`find`, `grep`, `grep -E`, pipes `|`

2. Unit I — command line, files, help, users

Navigation and file operations

```
pwd

cd /etc

ls -lah

cd /var/log

cd ~                    # home directory

mkdir -p project/{commands,reports,backup}
```

```
touch project/commands/{cmd1.txt,cmd2.txt,cmd3.txt}

cp source.txt backup/

mv old.txt new.txt

rm -i file.txt       # ask before deleting

rm -r directory      # recursive: use carefully
```

Finding files and shell expansion

```
find /etc -type f -name '*.conf' 2>/dev/null

find . -type f -iname '*report*'

ls data*.txt

ls report?.txt

# redirection

printf 'pwd\nls\nid\n' > linux_commands.txt

printf 'date\nwhoami\n' >> linux_commands.txt

cat linux_commands.txt
```

Text and documentation

```
less file.txt

head -n 10 file.txt

tail -f /var/log/messages

man chmod                 # search inside man: /pattern; quit: q

man -k password           # keyword search (same idea as apropos)

apropos permissions

info coreutils 'ls'
```

Users and groups

```
sudo groupadd developers

sudo useradd -m developer1

sudo useradd -m developer2

sudo usermod -aG developers developer1

sudo usermod -aG developers developer2

sudo passwd developer1

id developer1

groups developer1

getent group developers
```

```
sudo userdel -r developer1   # only when deletion is intended
```

Remember: `usermod -aG` means append supplementary groups. Without `-a`, existing supplementary groups can be replaced.

3. Unit II — permissions, processes, services, SSH, logs

Permissions and ownership

```
ls -l project.txt

chmod 640 project.txt       # owner rw, group r, others none

chmod 750 shared_lab        # owner rwx, group r-x, others none

chmod u+rw,g+r,o-rwx file

sudo chown user:group file

umask

# numeric: r=4, w=2, x=1; 640 = rw-r-----
```

Object	Read	Write	Execute
File	view contents	change contents	run as program
Directory	list names	create/delete entries	enter/traverse

Processes and jobs

```
ps -ef

ps -u "$USER"

top

pgrep -a process_name

sleep 300                 # Ctrl+Z suspends

jobs

bg %1

fg %1

kill PID                  # graceful TERM

kill -9 PID               # force only if necessary

systemctl status sshd
```

Services and SSH

```
sudo systemctl enable --now sshd
```

```
systemctl is-enabled sshd

systemctl is-active sshd

ssh user@localhost

whoami; hostname; pwd

exit

ssh-keygen -t ed25519

ssh-copy-id user@server

ssh user@server
```

Service name: RHEL commonly uses `sshd`. If unsure, use `systemctl list-unit-files | grep -i ssh`.

Logs and time

```
journalctl -n 50

journalctl -u sshd

journalctl -u sshd -r          # newest first

journalctl -p warning -b       # warnings/errors this boot

journalctl --since '1 hour ago'

systemctl status systemd-journald

timedatectl
```

4. Unit III — networking, archives, packages, filesystems

Network inspection and nmcli

```
nmcli device status

nmcli connection show

ip addr

ip route

ping -c 4 8.8.8.8

sudo nmcli con add con-name testing type ethernet ifname ens160

sudo nmcli con mod testing ipv4.method manual

sudo nmcli con mod testing ipv4.addresses 192.168.120.150/24

sudo nmcli con mod testing ipv4.gateway 192.168.120.2

sudo nmcli con mod testing ipv4.dns 8.8.8.8

sudo nmcli con up testing
```

Archives, packages, links

```
tar -czf backup.tar.gz directory/

tar -tzf backup.tar.gz

tar -xzf backup.tar.gz

scp file user@host:/tmp/

rsync -av directory/ user@host:/backup/

sudo dnf search keyword

sudo dnf install package

sudo dnf update

rpm -q package

rpm -ql package

ln -s target linkname

readlink linkname
```

Mounting filesystems

```
lsblk

sudo fdisk -l

sudo blkid

sudo mkdir -p /data

sudo mkfs.xfs /dev/sdb1       # destroys data on that partition

sudo mount /dev/sdb1 /data

df -hT

findmnt /data

sudo umount /data

# persistent mount: use UUID from blkid in /etc/fstab

UUID=XXXX  /data  xfs  defaults  0 0

sudo mount -a
```

Safety: Confirm the device with `lsblk` before `fdisk`, `mkfs`, `mount`, `umount`, or `mkswap`. A wrong device can erase data.

5. Unit IV — Bash, loops, regex, scheduling, performance, ACL

Bash script template

```
#!/bin/bash
```

```
read -p 'Marks: ' marks

if [ "$marks" -ge 40 ]; then

  echo 'PASS'

else

  echo 'FAIL'

fi

# save as check.sh, then:

chmod +x check.sh

./check.sh
```

Loops and conditions

```
for n in 1 2 3; do echo "$n"; done

for f in *.txt; do echo "$f"; done

n=1

while [ "$n" -le 5 ]; do echo "$n"; n=$((n+1)); done

if [ -f file.txt ]; then echo exists; fi

[ "$a" -gt "$b" ] && echo greater
```

Regular expressions and grep

```
grep '^ERROR' app.log       # beginning of line

grep 'failed$' app.log      # end of line

grep -E 'error|failed' app.log

grep -E '^[0-9]{5}$' file.txt

grep -i 'warning' log.txt

grep -v '^#' config.txt       # exclude comments
```

Regex	Meaning
`^`, `$`	start, end of line
`.`	any one character
`[abc]`, `[0-9]`	one character from a set/range
`*`, `+`, `?`	zero-or-more, one-or-more, optional (extended regex)
`{n}`, `{n,m}`	exact/ranged repetition with `grep -E`

Scheduling and ACL

```
at now + 5 minutes

# type commands, then Ctrl+D

atq

at -c JOB_ID

atrm JOB_ID

crontab -e

# minute hour day month weekday command

setfacl -m u:alice:rw file

getfacl file

setfacl -m g:developers:rX directory

setfacl -b file
```

6. Unit V — SELinux, partitions, swap, LVM, Stratis/VDO

SELinux quick checks

```
getenforce

sestatus

sudo setenforce 0       # temporary permissive; use only when instructed

ls -Z

sudo restorecon -Rv /var/www/html

getsebool -a

sudo ausearch -m AVC -ts recent
```

Partitions, swap, and LVM concepts

```
lsblk

sudo fdisk /dev/sdb

sudo mkfs.xfs /dev/sdb1

sudo mkswap /dev/sdb2

sudo swapon /dev/sdb2

swapon --show

sudo swapoff /dev/sdb2

# LVM conceptual workflow:

sudo pvcreate /dev/sdb1
```

```
sudo vgcreate vgdata /dev/sdb1

sudo lvcreate -n lvdata -L 2G vgdata

sudo mkfs.xfs /dev/vgdata/lvdata

sudo mount /dev/vgdata/lvdata /data

sudo lvextend -r -L +1G /dev/vgdata/lvdata
```

Persistent configuration: use UUIDs in `/etc/fstab`; test with `mount -a`. For swap, the fstab entry uses `none swap defaults 0 0`.

7. Unit VI — NFS, boot, firewall, installation

```
# NFS client examples

showmount -e nfs-server

sudo mkdir -p /mnt/share

sudo mount -t nfs nfs-server:/export /mnt/share

findmnt /mnt/share

# automount is normally configured with autofs and /etc/auto.master


# boot and troubleshooting

systemctl get-default

sudo systemctl set-default multi-user.target

journalctl -b

# firewall

sudo firewall-cmd --state

sudo firewall-cmd --list-all

sudo firewall-cmd --permanent --add-service=http

sudo firewall-cmd --reload

# SELinux port labels

semanage port -l | grep ssh

sudo semanage port -a -t ssh_port_t -p tcp 2222
```

8. Fifteen practical scenarios — model answers

1. `pwd; cd /etc; ls -l; cd /var/log; cd ~`
2. `find /etc -type f -name '*.conf' 2>/dev/null`
3. `mkdir -p ~/LinuxAssessment/{commands,reports,backup}; touch`

`~/LinuxAssessment/commands/cmd{1,2,3}.txt; find ~/LinuxAssessment -print`

4. `touch data{1,2,3}.txt report{1,2}.txt; printf '%s\n' data*.txt; printf '%s\n' report*.txt`
5. `printf 'pwd\nls\nid\n' > linux_commands.txt; cat linux_commands.txt; printf 'date\nwhoami\n' >> linux_commands.txt`
6. `man chmod`; search inside with `/OPTIONS`; installed keyword search with `apropos keyword` or `man -k keyword`.
7. `sudo groupadd developers; sudo useradd -m developer1; sudo useradd -m developer2; sudo usermod -aG developers developer1 developer2; getent group developers`
8. `sudo useradd -m labuser; sudo passwd labuser; id labuser; getent passwd labuser`
9. `touch project.txt; chmod 640 project.txt; stat -c '%A %a %n' project.txt`
10. `mkdir shared_lab; chmod 750 shared_lab; stat -c '%A %a %n' shared_lab`
11. Run `sleep 300`; press `Ctrl+Z`; run `bg; jobs; fg`.
12. `ps -u "$USER"; kill PID; ps -p PID`
13. `systemctl status sshd; sudo systemctl enable sshd; systemctl is-enabled sshd`
14. `ssh "$USER"@localhost; whoami; hostname; pwd; exit`
15. `journalctl -n 50; journalctl -u sshd; journalctl -u sshd -r; journalctl -p warning -b`

9. Last-minute checklist

- Can you explain the permission digits 640 and 750?
- Can you distinguish `find` from `grep`?
- Can you use `systemctl` to inspect, start, stop, enable, and verify a service?
- Can you inspect a network using `nmcli`, `ip`, and `ping`?
- Can you create a filesystem, mount it, and make it persistent using UUID/fstab?
- Can you explain SELinux enforcing/permissive and read an AVC denial?
- Can you write an `if` script and a `for`/`while` loop?
- Can you schedule one deferred job with `at` and recurring jobs with `cron`?
- For every command, can you state how you will verify the result?

End of revision guide · Use the original presentations and syllabus for deeper explanations.

CSE 493
LINUX SYSTEM ADMINISTRATION

Unit I

Accessing the command line : Accessing command line using local console, Accessing command line using desktop, Executing commands using bash shell

Managing Files From the Command Line : The Linux File System Hierarchy, Locating Files by Name, Managing Files Using Command-Line Tools, Matching File Names Using Path Name Expansion, Managing Files with Shell Expansion

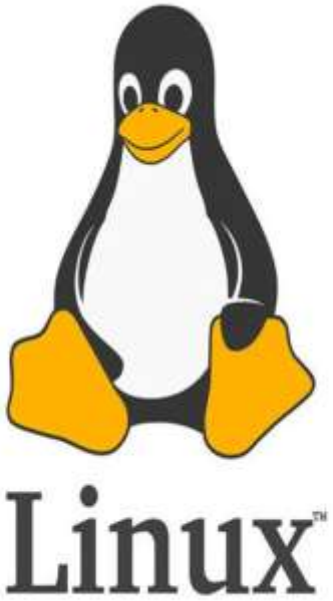
Getting Help in Red Hat Enterprise Linux : Reading Documentation Using man Command,
Reading Documentation Using pinfo Command, Reading Documentation in /usr/share/doc, Getting
Help From Red Hat

Creating, Viewing, and Editing Text Files : Redirecting Output to a File or Program, Editing Text
Files from the Shell Prompt, Editing Text Files with a Graphical Editor

Managing Local Linux Users and Groups : Users and Groups, Gaining Superuser Access,
Managing Local User Accounts, Managing Local Group Accounts, Managing User Passwords

WHAT IS LINUX?

Linux® is an open source operating system (OS) created by Linus Torvalds in 1991. Today, it has a massive user base, and is used in the world's 500 most powerful supercomputers. Users gravitate toward it for its versatility and security capabilities, among other reasons.



Creation



WHY SHOULD YOU LEARN ABOUT LINUX ?

1. You are probably already interacting with Linux systems in your daily life
2. Linux is a critical technology for IT professionals to understand
3. In application development, Linux hosts the application or its runtime.
4. In cloud computing, the cloud instances in the private or public cloud environment use Linux as the operating system.
5. With mobile applications or the Internet of Things (IoT), the chances are high that the operating system of your device uses Linux.
6. If you are looking for new opportunities in IT, Linux skills are in high demand.

WHAT MAKES LINUX GREAT?

Freedom. Power. Flexibility.

1. OPEN SOURCE SOFTWARE

You can see it.
You can change it.
You can share it.



Open source means innovation happens faster.
The community improves it, and everyone benefits.

2. POWERFUL & SCRIPTABLE COMMAND-LINE INTERFACE

Do anything.
Automate everything.
From anywhere.



Built for admins.
Made for automation.
Perfect for scale.

3. MODULAR & FLEXIBLE

Use what you need.
Replace what you want.
Remove what you don't.



Lightweight & minimal

Full-featured & powerful

From a tiny appliance to an enterprise powerhouse—
Linux adapts to you.

LINUX

Built to empower.
Designed to evolve.
Free to innovate.

THAT'S THE POWER OF LINUX. Open. Powerful. Flexible.
It's not just an operating system. It's a movement.

Accessing the command line

OBJECTIVES

After completing this section, you should be able to log in to a Linux system and run simple commands using the shell.

INTRODUCTION TO THE BASH SHELL :

A command line is a text-based interface which can be used to input instructions to a computer system. The Linux command line is provided by a program called the shell

When a shell is used interactively, it displays a string when it is waiting for a command from the user. This is called the shell prompt. When a regular user starts a shell, the default prompt ends with a **$** character, as shown below :

```
[user@host ~]$
```

```
[root@host ~]#
```

Can you Spot the difference over here?

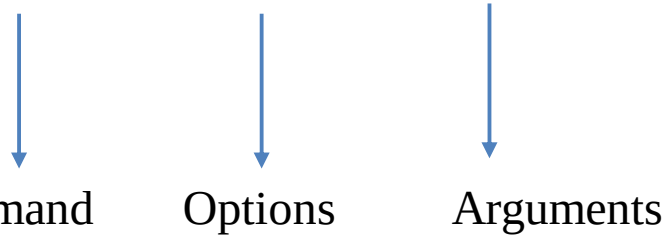
Root User	Regular User
1. Full Access: Can access and modify every file and folder in the system.	**Limited Access:** Can access only their own files and permitted resources.
2. Administrative Tasks: Can install software, create/delete users, and change system settings.	**Cannot perform administrative tasks** unless given permission using sudo.
3. Prompt Symbol: The terminal prompt ends with **#**. Example: root@ubuntu:~#	**Prompt Symbol:** The terminal prompt ends with **$**. Example: User@ubuntu:~$
4. Risk Level: A wrong command can damage the entire operating system.	**Safer:** Mistakes usually affect only the user's own files and account.

SHELL BASICS

Commands entered at the shell prompt have three basic parts:

- Command to run
- Options to adjust the behavior of the command.
- Arguments, which are typically targets of the command

E.g **Usermod -L user01**



LOGGING IN TO A LOCAL COMPUTER

To run the shell, you need to log in to the computer on a terminal. A terminal is a text-based interface used to enter commands into and print output from a computer system. There are several ways to do this.

Physical Console
Directly connected keyboard + monitor plugged into the machine

Virtual Consoles (tty1–tty6)
One physical console → 6 independent login sessions Switch between them: **Ctrl+Alt+F1** to **F6**

Two Types of Logins

Console	Type	What You Get
tty1	Graphical login screen	Desktop environment
tty2–tty6	Text login prompt	Shell prompt directly

LOGGING IN OVER THE NETWORK

Linux users and administrators often need to get shell access to a remote system by connecting to it over the network. In a modern computing environment, many headless servers are actually virtual machines or are running as public or private cloud instances

The most common way to get a shell prompt on a remote system is to use Secure Shell (SSH).

```
[user@host ~]$ ssh remoteuser@remotehost
remoteuser@remotehost's password: password
[remoteuser@remotehost ~]$
```

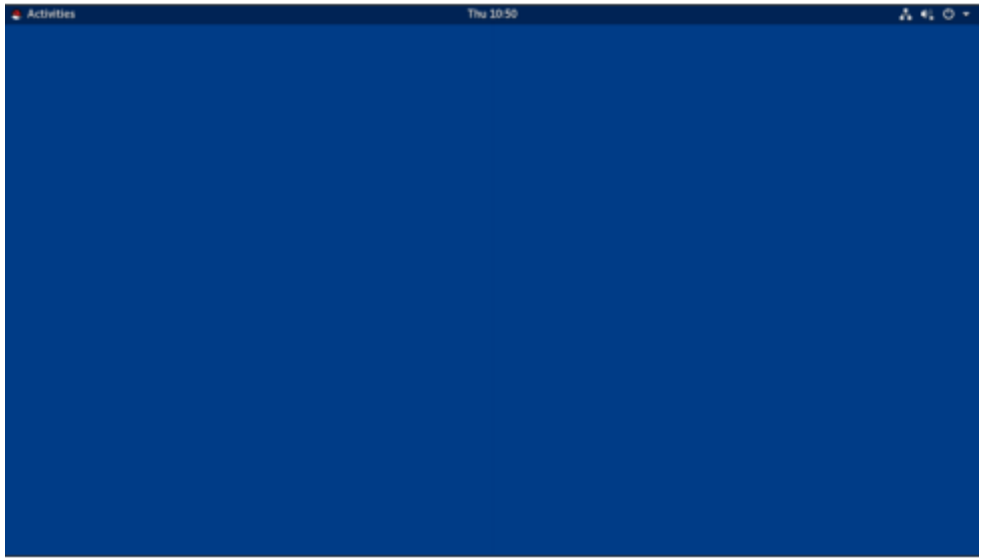
The **ssh** command encrypts the connection to secure the communication against eavesdropping or hijacking of the passwords and content.

LOGGING OUT : When you are finished using the shell and want to quit **Ctrl +D**

ACCESSING THE COMMAND LINE USING THE DESKTOP

The desktop environment is the graphical user interface on a Linux system. The default desktop environment in Red Hat Enterprise Linux 8 is provided by GNOME 3. It provides an integrated desktop for users and a unified development platform on top of a graphical framework provided by either Wayland (by default) or the legacy Window System.

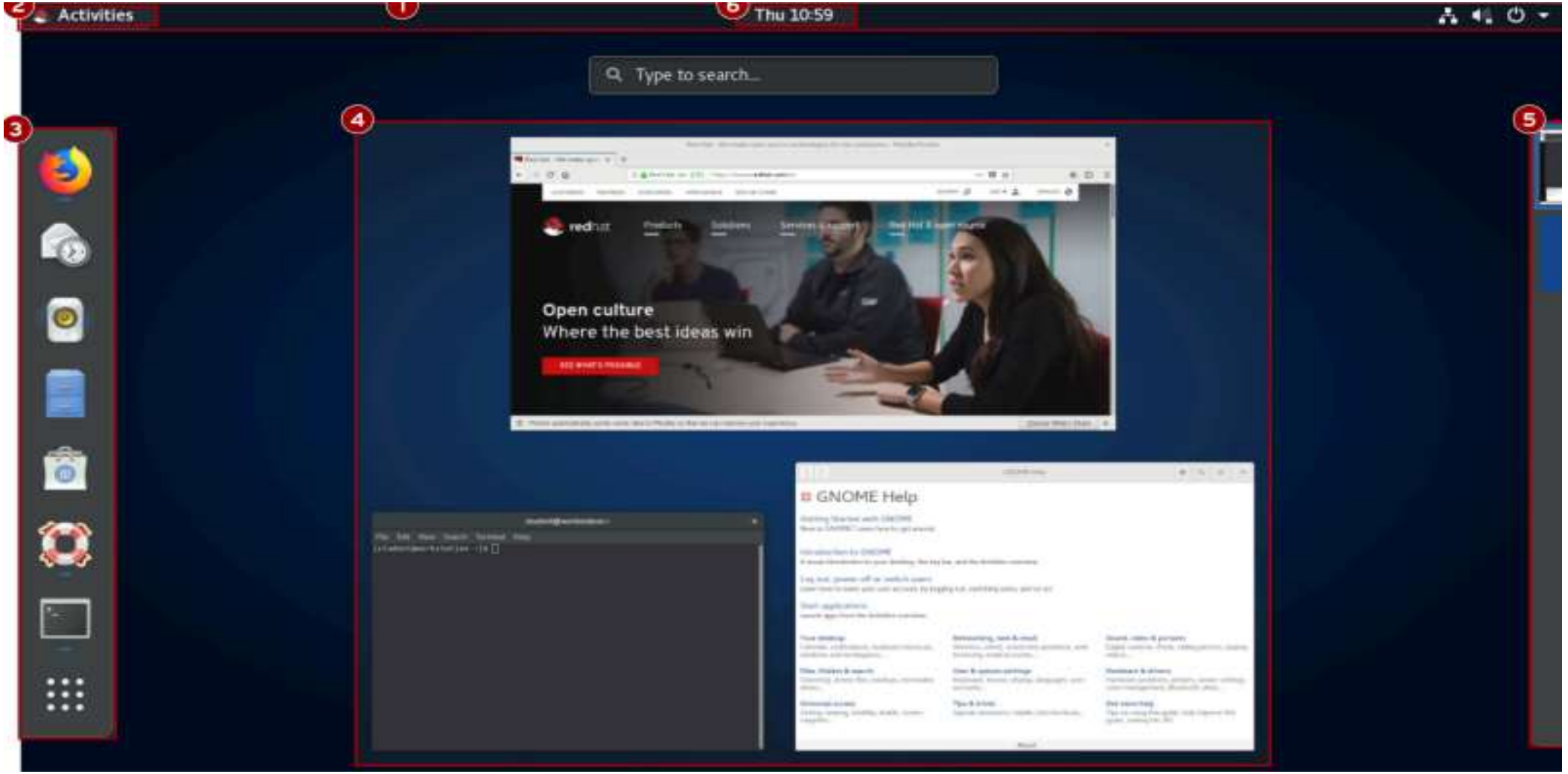
1. **GNOME Shell** is the main interface of the GNOME desktop — and it's customizable.
2. **RHEL 8** uses the **"Standard"** theme by default; **RHEL 7** used **"Classic"** (older GNOME look).
3. To switch themes: click the **gear icon** ⚙ next to "Sign In" — **before** entering your password.
4. Your chosen theme is **saved** and used every time you log in.



Empty GENOME Desktop

You can quickly start GNOME Help by clicking the Activities button on the left side of the top bar, and in the dash that appears on the left side of the screen, clicking the life ring buoy icon to launch it.

ACCESSING THE COMMAND LINE USING THE DESKTOP



ACCESSING THE COMMAND LINE USING THE DESKTOP

1. **Top Bar:** The bar at the top of the screen that provides access to Activities, system status, settings, notifications, and user options.
2. **Activities Overview:** A mode for launching applications, viewing open windows, and managing workspaces.
3. **Dash (Dock):** A sidebar containing favorite apps, running applications, and the Applications Grid button. **Windows Overview:** Displays thumbnails of all open windows in the current workspace for easy switching and organization.
4. **Workspace Selector:** Shows available workspaces and allows you to switch between them or move windows across workspaces.

WORKSPACES

Workspaces are separate desktop screens that have different application windows. These can be used to organize the working environment by grouping open application windows by task. For example, windows being used to perform a particular system maintenance activity (such as setting up a new remote server) can be grouped in one workspace, while email and other communication applications can be grouped in another workspace.

There are two simple methods for switching between workspaces. One method, perhaps the fastest, is to press **Ctrl**+**Alt**+**UpArrow** or **Ctrl**+**Alt**+**DownArrow** to switch between workspaces sequentially. The second is to switch to the Activities overview and click the desired
workspace.

STARTING A TERMINAL

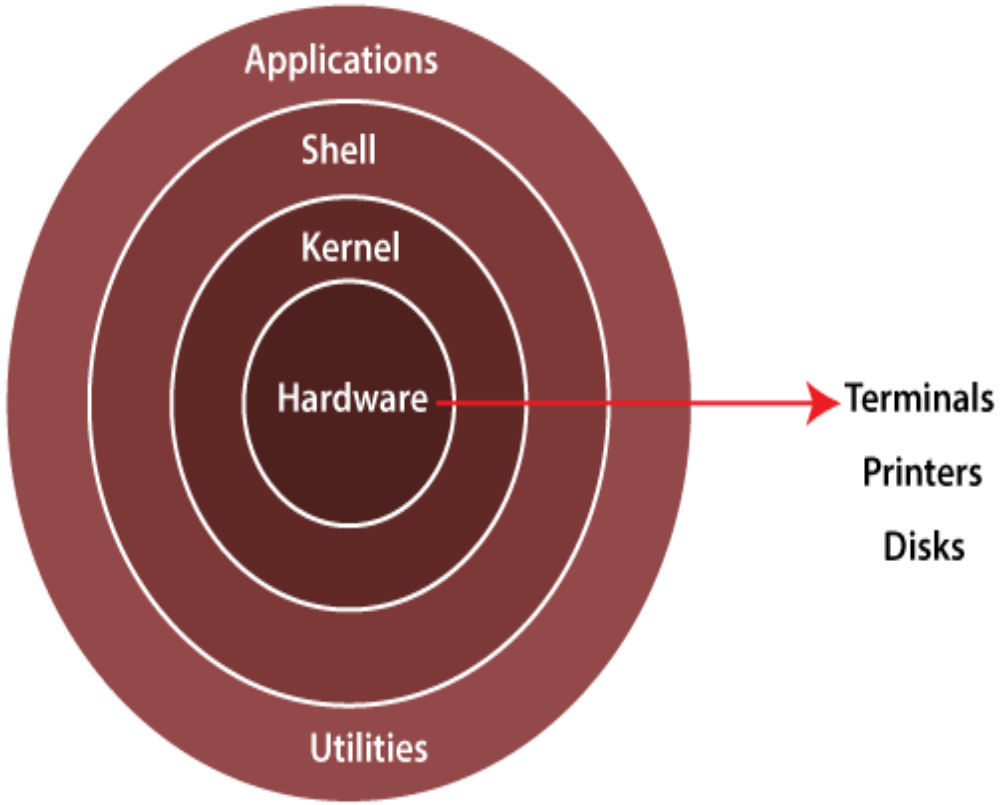
To get a shell prompt in GNOME, start a graphical terminal application such as GNOME Terminal. There are several ways to do this. The two most commonly used methods are listed below:

- From the Activities overview, select Terminal from the dash (either from the favorites area or by finding it with either the grid button (inside Utilities grouping) or the search field at the top of the windows overview).
- Press the **Alt**+**F2** key combination to open the Enter a Command and enter **gnome-terminal**.

LOCKING THE SCREEN OR LOGGING OUT

To lock the screen, from the system menu in the upper-right corner, click the lock button at the bottom of the menu or press **Super**+**L** (which might be easier to remember as **Windows**+**L**).

EXECUTING COMMANDS USING THE BASH SHELL



Linux Architecture

Kernel is the core component of the Linux operating system that sits between the hardware and user space, managing system resources and ensuring smooth communication between software and hardware.

Shell and Utilities

Command interpreter: A user interface (such as Bash) that accepts text commands and passes them along to be executed.

System tools: Small programs used by administrators to configure, monitor, and maintain the system

Programs: The topmost layer where user-facing software, office tools, and graphical or command-line programs run isolated from direct hardware access for enhanced stability

EXECUTING COMMANDS USING THE BASH SHELL

```
[user@host]$ whoami
user
[user@host]$
```

whoami: Displays the username of the currently logged-in user.
Output: user → The current logged-in user's username is **user**.

```
[user@host]$ command1;command2
```

If you want to type more than one command on a single line, use the semicolon (**;**) as a command separator

Simple Example

```
[user@host]$ pwd; date
/home/user
Mon Jul 27 10:45:30 IST 2026
[user@host]$
```

EXECUTING COMMANDS USING THE BASH SHELL

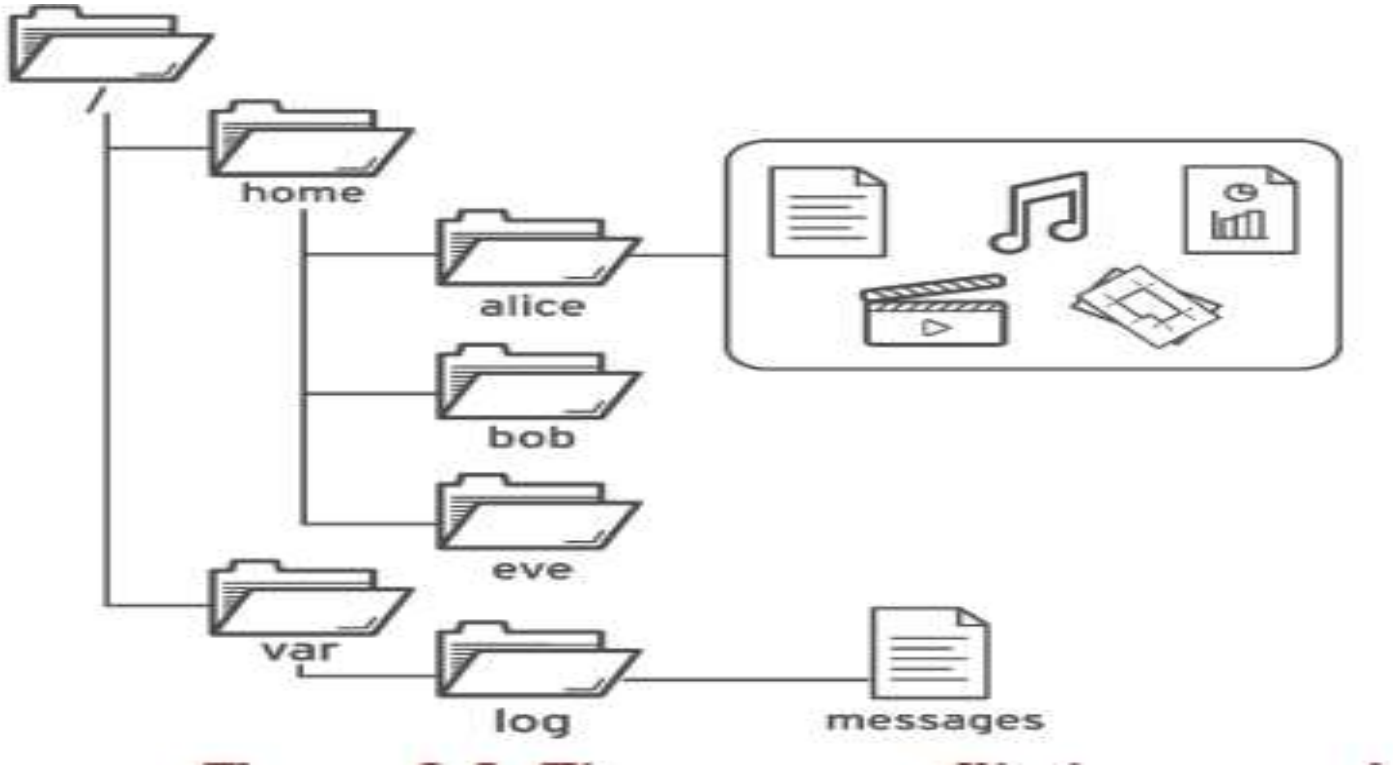
```
[user@host ~]$ date
Sat Jan 26 08:13:50 IST 2019
[user@host ~]$ date +%R
08:13
[user@host ~]$ date +%x
01/26/2019
```

The **date** command displays the current date and time. It can also be used by the superuser to set the system clock. An argument that begins with a plus sign (+) specifies a format string for the date command.

```
[user@host ~]$ passwd
Changing password for user user.
Current password: old_password
New password: new_password
Retype new password: new_password
passwd: all authentication tokens updated successfully.
```

The **passwd** command changes a user's own password. The original password for the account must be specified before a change is allowed. The superuser can use the **passwd** command to change other users' passwords

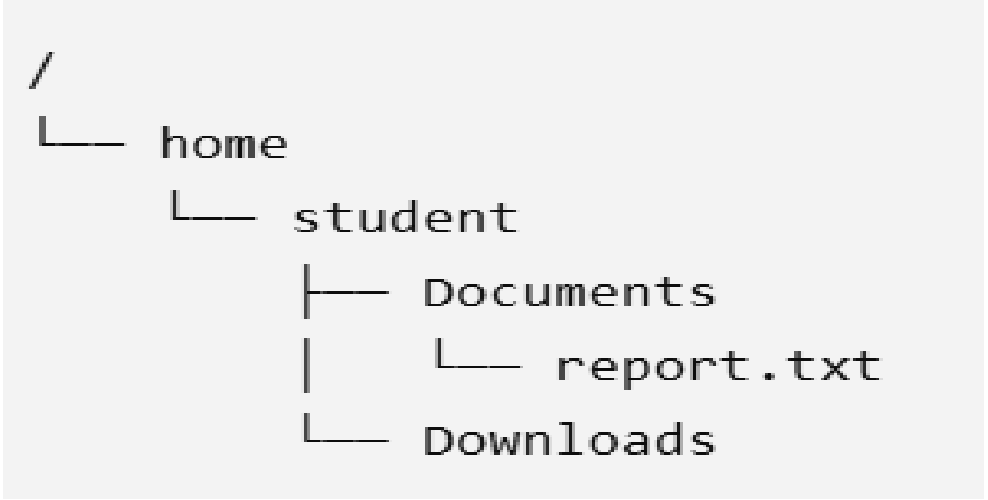
Absolute Path vs Relative Path



Absolute Path vs Relative Path

Feature	Absolute Path	Relative Path
1. Starting Point	Starts from the **root directory (/)**	Starts from your **current working directory**
2. Begins With	Always begins with **/**	Never starts with /. It may start with ./, ../, or a folder name.
3. Depends on Current Location?	No. It works from anywhere in the filesystem.	Yes. It only works correctly if you are in the expected current directory.
4. Example	/home/student/Documents/report.txt	Documents/report.txt, ../Documents/report.txt, ./report.txt

Absolute Path vs Relative Path



Suppose your current directory is Downloads
-> To open report.txt

Absolute Path will be -> cat /home/student/Documents/report.txt

Relative Path will be -> cat ../Documents/report.txt (..) Why?? Because you are currently in Downloads

Frequently used Commands

ls	Lis the contents of the fie	ls ,ls –l , ls -a
cd	Changes the current working directory.	cd /home/user/projects
pwd	Displays the full path of the current working directory.	pwd
mkdir	Creates a new directory.	mkdir my_project
rm	Removes files or directories. Use -r for directories.	rm file.txt or rm -r old_folder
cp	Copies files or directories from one location to another.	cp file.txt backup.txt
mv	Moves or renames files and directories.	mv old.txt new.txt
cat	Displays the contents of a file or combines multiple files.	cat README.md



Command Line

Description

aksh_maverick → Current logged-in user.
AKSHANSH-PC → Computer (host) name.
~ → Current directory is the user's home directory.
$ → Indicates a normal user shell is ready to accept commands.

```
aksh_maverick@AKSHANSH-PC:~$ mkdir schoolai schoolcse schoolece
aksh_maverick@AKSHANSH-PC:~$ ls
3                                  course1    Downloads_backup  myorg-validator.pem  Project1   schoolece  user1
chef-server-core_15.1.7-1_amd64.deb.1  course2    linux_for_devops  newfile.txt          Project2   snap       user2
chef-workstation.deb               Developer  LOPEZ             newuser              Projects   Technova   user3
Classroom1                         devops     love              Practical1           schoolai   test.sh
course                             Downloads  MynewProject      project              schoolcse  Thesis
aksh_maverick@AKSHANSH-PC:~$
```

Command Line

Description

Command/Action	Description
mkdir schoolai schoolcse schoolece	Creates three directories (schoolai, schoolcse, and schoolece) in a single command.
ls	Lists all files and directories in the current working directory (~).

```
aksh_maverick@AKSHANSH-PC:~$ mkdir schoolai schoolcse schoolece
aksh_maverick@AKSHANSH-PC:~$ ls
3                                  course1    Downloads_backup  myorg-validator.pem  Project1   schoolece  user1
chef-server-core_15.1.7-1_amd64.deb.1  course2    linux_for_devops  newfile.txt          Project2   snap       user2
chef-workstation.deb               Developer  LOPEZ             newuser              Projects   Technova   user3
Classroom1                         devops     love              Practical1           schoolai   test.sh
course                             Downloads  MynewProject      project              schoolcse  Thesis
aksh_maverick@AKSHANSH-PC:~$
```

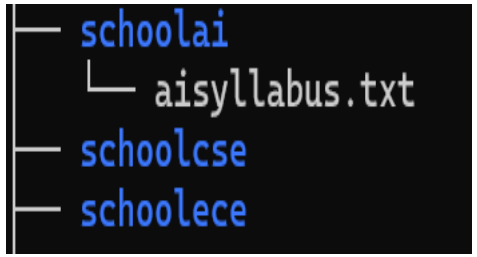
Command Line

Description

Command/Action	Description
mkdir schoolai schoolcse schoolece	Creates three directories (schoolai, schoolcse, and schoolece) in a single command.
ls	Lists all files and directories in the current working directory (~).

```
aksh_maverick@AKSHANSH-PC:~/schoolai$ echo "Below is the attached syllabus " >aisyllabus.txt
aksh_maverick@AKSHANSH-PC:~/schoolai$ cat aisyllabus.txt
Below is the attached syllabus
aksh_maverick@AKSHANSH-PC:~/schoolai$ ls
aisyllabus.txt
aksh_maverick@AKSHANSH-PC:~/schoolai$ cd ..
aksh_maverick@AKSHANSH-PC:~$ tree
```

Tree Structure like display



Command/Action	Description
echo "Below is the attached syllabus" > aisyllabus.txt	Creates a new file named aisyllabus.txt and writes the given text into it, replacing any existing content.
cat aisyllabus.txt	Displays the contents of the aisyllabus.txt file on the terminal, confirming the text was written successfully.
ls and cd ..	ls lists the file in the current directory, while cd .. moves one level up to the parent directory before running tree to view the directory structure.

```
aksh_maverick@AKSHANSH-PC:~$ pwd
/home/aksh_maverick
aksh_maverick@AKSHANSH-PC:~$ cd schoolcse
aksh_maverick@AKSHANSH-PC:~/schoolcse$ touch syllabuscse.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ ls
syllabuscse.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ touch listofstudents.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ ls
listofstudents.txt  syllabuscse.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ pwd
/home/aksh_maverick/schoolcse
aksh_maverick@AKSHANSH-PC:~/schoolcse$ cp listofstudents.txt ../schoolai/
aksh_maverick@AKSHANSH-PC:~/schoolcse$ ls
listofstudents.txt  syllabuscse.txt
aksh_maverick@AKSHANSH-PC:~/schoolcse$ cd ..
aksh_maverick@AKSHANSH-PC:~$ cd schoolai
aksh_maverick@AKSHANSH-PC:~/schoolai$ ls
aisyllabus.txt  listofstudents.txt
aksh_maverick@AKSHANSH-PC:~/schoolai$ pwd
/home/aksh_maverick/schoolai
aksh_maverick@AKSHANSH-PC:~/schoolai$
```

Step	Description
1. Navigate and create files	The user checks the current directory using pwd, enters the schoolcse directory, and creates two files: syllabuscse.txt and listofstudents.txt using the touch command.
2. Verify the files	The ls command is used to confirm that both files have been successfully created in the schoolcse directory.
3. Copy file using a relative path	The command cp listofstudents.txt ../schoolai/ copies the listofstudents.txt file from schoolcse to the sibling directory schoolai using the relative path ../schoolai/.
4. Confirm the copy	The user navigates to the schoolai directory and uses ls and pwd to verify that listofstudents.txt has been copied successfully and to display the current directory path.

```
aksh_maverick@AKSHANSH-PC:~$ mkdir FindDemo
cd FindDemo

mkdir Documents Pictures Music

touch Documents/report.txt
touch Documents/notes.txt
touch Pictures/photo.jpg
touch Pictures/logo.png
touch Music/song.mp3
aksh_maverick@AKSHANSH-PC:~/FindDemo$ find . -name "report.txt"
./Documents/report.txt
aksh_maverick@AKSHANSH-PC:~/FindDemo$ find . -name "*.txt"
./Documents/notes.txt
./Documents/report.txt
aksh_maverick@AKSHANSH-PC:~/FindDemo$ find . -type d
.
./Pictures
./Documents
./Music
aksh_maverick@AKSHANSH-PC:~/FindDemo$ find Documents -name "*.txt"
Documents/notes.txt
Documents/report.txt
aksh_maverick@AKSHANSH-PC:~/FindDemo$ ls
Documents  Music  Pictures
aksh_maverick@AKSHANSH-PC:~/FindDemo$ pwd
/home/aksh_maverick/FindDemo
aksh_maverick@AKSHANSH-PC:~/FindDemo$
```

Option	Meaning	Example
.	Search from the current directory.	find . -name "*.txt"
-name	Search by file or directory name.	find . -name "report.txt"
"*.txt"	* matches any number of characters before .txt.	notes.txt, report.txt
-type f	Search only for regular files.	find . -type f
-type d	Search only for directories.	find . -type d

Scenario

You have recently joined **LPU Solutions** as a **Junior Linux System Administrator**.

The company has started a new software development project, but the project workspace is not yet organized. Your team lead has assigned you the responsibility of setting up the directory structure, managing files, organizing departmental data, locating important documents, and cleaning up unnecessary files using only Linux command-line tools.

Task 1

Verify your current working directory and examine the existing contents.

Task 2

Create a directory structure for the following departments:

HR Sales IT Finance Projects

Task 3

Create the required files for each department according to their work responsibilities.

Task 4

Insert appropriate sample data into all text files and verify the contents.

Task 6

Copy selected files from one department to another while keeping the original files unchanged.

Task 7

Locate specific files whose locations are unknown without manually browsing the directory structure.

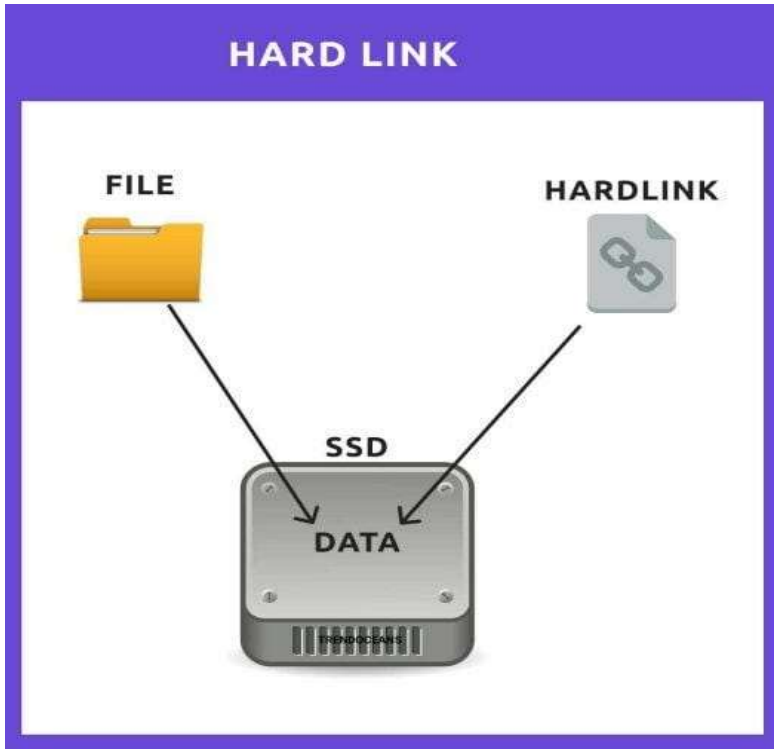
Task 8

Delete unnecessary files, remove empty directories, and clean up obsolete project folders.

What are Hard Links and Soft Links in Linux

Hard Link

The concept of a hard link is the most basic we will discuss today. Every file on the Linux filesystem starts with a single hard link. The *link* is between the filename and the actual data stored on the filesystem



How to Create a Hard Link

aksh_maverick@AKSHANSH-PC: echo “text” >file_name.txt
aksh_maverick@AKSHANSH-PC: ln file_name.txt file_name2.txt

If you edit one file ?

The changes will be reflected in both files

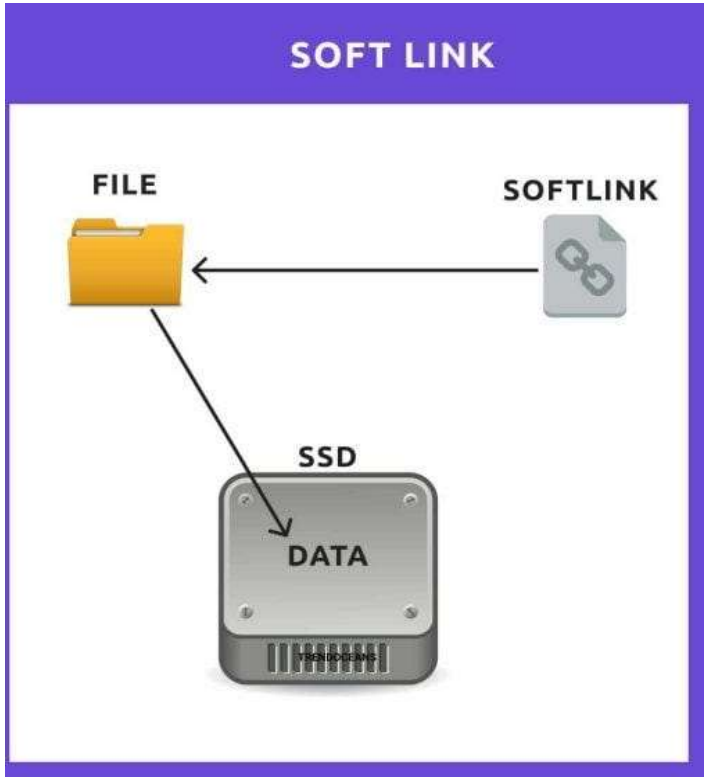
Now comes the concept of Inode

Since both are pointing to the same file they will have same inode number

What are Hard Links and Soft Links in Linux

Soft Link

A **soft link** in Linux is a special file that acts as a shortcut by **pointing to the text path of another file or directory**. Unlike a hard link, it has a unique inode number and can cross different file systems, but it breaks if the original file is deleted (ln -s file_name link_name)



How to Create a soft Link?

```
aksh_maverick@AKSHANSH-PC:~$ echo "This is myfile " > d1.txt
aksh_maverick@AKSHANSH-PC:~$ echo "this is updated file" > d2.txt
aksh_maverick@AKSHANSH-PC:~$ cat d2.txt
this is updated file
aksh_maverick@AKSHANSH-PC:~$ cat d1.txt
this is updated file
aksh_maverick@AKSHANSH-PC:~$ echo "This is my notes1.txt" > notes1.txt
aksh_maverick@AKSHANSH-PC:~$ ln -s notes1.txt shortcut.txt
aksh_maverick@AKSHANSH-PC:~$ cat shortcut.txt
This is my notes1.txt
aksh_maverick@AKSHANSH-PC:~$ rm notes1.txt
aksh_maverick@AKSHANSH-PC:~$ cat shortcut.txt
cat: shortcut.txt: No such file or directory
aksh_maverick@AKSHANSH-PC:~$
```

What is the difference betweem Hard Links and Soft Links in Linux

Feature	Hard Link	Soft Link (Symbolic Link)
Concept	Another **name** for the same file.	A **shortcut** that stores the path to the original file.
If Original File is Deleted	Still works because the data is still accessible through the hard link.	Becomes a **broken (dangling) link** because the target file no longer exists.
Directories	Cannot normally be created for directories.	Can be created for both **files and directories**.
Different File Systems	Cannot be created across different file systems or partitions.	Can be created across different file systems or partitions.

What is Output Redirection?

Normally, every command displays its output on the **terminal**.

What if I want to save its output?

```
aksh_maverick@AKSHANSH-PC:~$ date > date.txt
aksh_maverick@AKSHANSH-PC:~$ cat date.txt
Thu Jul 30 11:13:24 AM IST 2026
aksh_maverick@AKSHANSH-PC:~$
```

Rule 1: > Overwrite

```
aksh_maverick@AKSHANSH-PC:~$ echo "Linux Commands" > a1.txt
aksh_maverick@AKSHANSH-PC:~$ cat a1.txt
Linux Commands
aksh_maverick@AKSHANSH-PC:~$ echo "Linux Commands Updated" > a1.txt
aksh_maverick@AKSHANSH-PC:~$ cat a1.txt
Linux Commands Updated
aksh_maverick@AKSHANSH-PC:~$
```

If the file already exists, its old content is **deleted** and replaced.

What is Output Redirection?

Normally, every command displays its output on the **terminal**.

Rule 2: >> Append

```
aksh_maverick@AKSHANSH-PC:~$ echo "Linux Commands"> a1.txt
aksh_maverick@AKSHANSH-PC:~$ cat a1.txt
Linux Commands
aksh_maverick@AKSHANSH-PC:~$ echo "Linux Commands Updated">>a1.txt
aksh_maverick@AKSHANSH-PC:~$ cat a1.txt
Linux Commands
Linux Commands Updated
aksh_maverick@AKSHANSH-PC:~$
```

Adds new data to the end.

What is Output Redirection?

Normally, every command displays its output on the **terminal**.

Rule 3: 2> Store Errors

```
aksh_maverick@AKSHANSH-PC:~$ ls abc.txt
ls: cannot access 'abc.txt': No such file or directory
aksh_maverick@AKSHANSH-PC:~$ ls abc.txt 2> errors.txt
aksh_maverick@AKSHANSH-PC:~$ cat erros.txt
cat: erros.txt: No such file or directory
aksh_maverick@AKSHANSH-PC:~$
```

This stores errros in a file.

What is Pipeline?

A **pipeline** sends the **output of one command** directly as the **input of another command**. Denoted by symbol **(|)**

Why Do We Need Pipelines?

Scenario : Suppose you have 500 files. Someone asks you Just tell me how many files are there

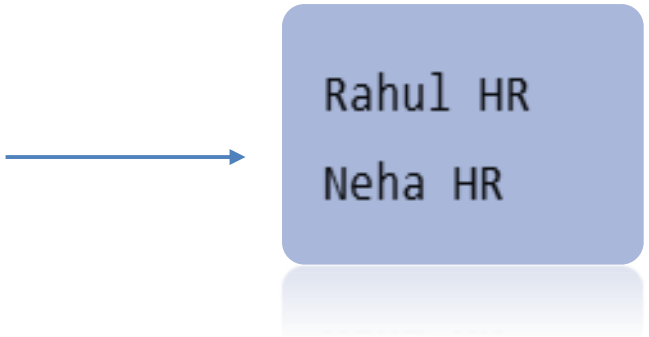
Your Approach
Run Command ls and count manually

Using Pipeline :
ls | wc -l

What is Pipeline?

Find Only HR Employees
Suppose employees.txt contains:
Rahul HR
Priya Sales
Aman IT
Neha HR

Some one Asks?
"Show me only HR employees.“
cat employees.txt | grep HR



Scenario

You have joined **LPU solutions.** as a **Junior Linux System Administrator**. Your Team Lead has assigned you a practical test to evaluate your Linux command-line skills

Part 1 – Hard Link
Create a hard link of employees.txt inside the backup directory.
Verify that both files point to the same inode.

Part 2 - Soft Link
Create a symbolic (soft) link of budget.txt inside the reports directory.
Verify that the symbolic link has been created successfully.

Part 3 – Input and Output Redirection
Save the current date and time into a file named daily_report.txt.
Append the current logged-in user's name to the same file.

Part 4 – Pipelines
Display only the lines containing the word ERROR from server.log.

EDITING TEXT FILES FROM THE SHELL PROMPT

A key design principle of Linux is that information and configuration settings are commonly stored in text-based files. These files can be structured in various ways, as lists of settings, as structured XML or YAML, and so on. However, the advantage of text files is that they can be viewed and edited using any simple text editor

EDITING FILES WITH VIM

1. **Edit configuration files:** Use a text editor directly from a text-only shell to create, modify, and save text-based configuration files.
2. **Work on local or remote systems:** Edit files efficiently from a terminal window or during remote administration using **SSH** or the **Web Console**, even without a graphical interface.

1. Command Mode (Default Mode)

```
aksh_maverick@AKSHANSH-PC:~$ vim notes.txt
```



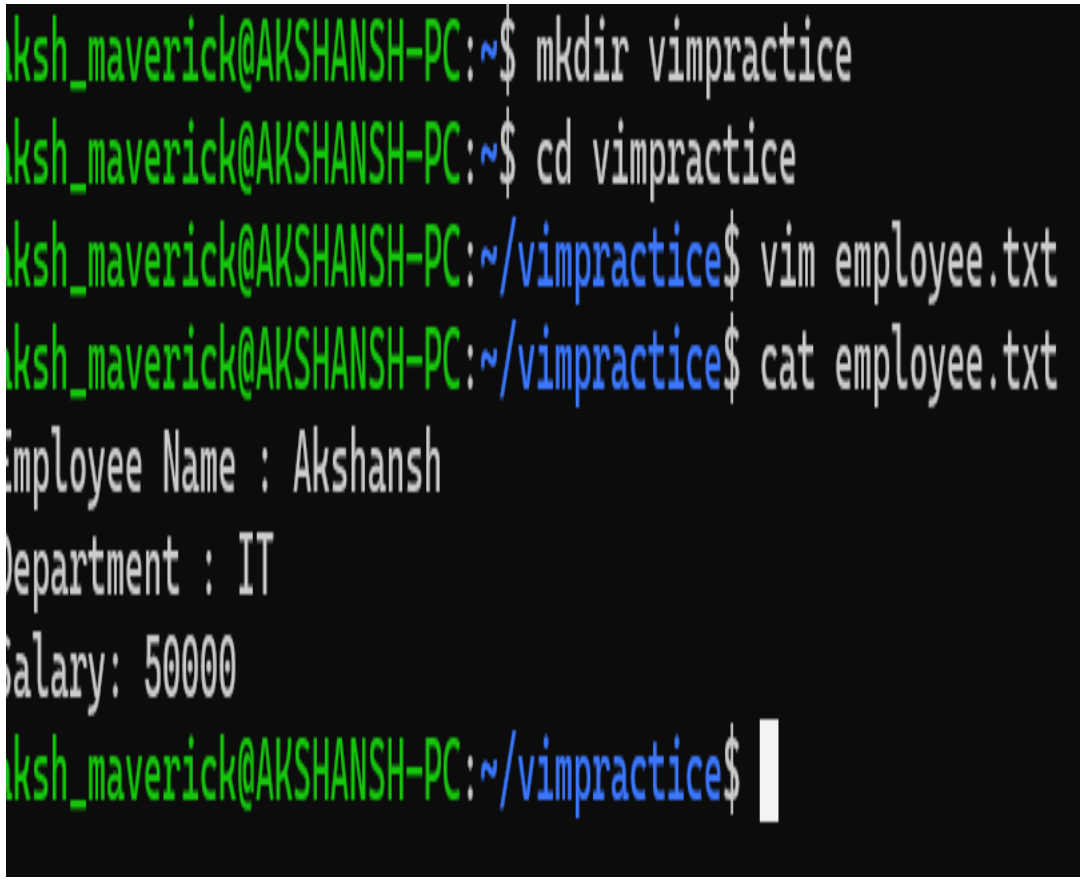


2. Insert Mode (I)

When you Press I you get to type becomes text**Note – When you finish Press ESC to get back to command mode**

.

3. **Visual Mode (V)**
Then you can
->Copy
->Delete
->Move
Character selection : **Shift +V**
Whole line selection : **Ctrl+V**
4. Extended Command Mode
Save - :w
Quit - :q
Save and quit - :wq



CHANGING THE SHELL ENVIRONMENT

USING SHELL VARIABLES

1. **Shell variables** are used to store values (such as text, paths, or numbers) that can be reused in commands or to change the shell's behavior.
2. They help reduce typing by storing long commands or frequently used values.
3. A **shell variable** exists **only in the current shell session**.
4. If you open another terminal or log in again, it gets a **separate shell** with its **own set of shell variables**.
5. You can **export** a shell variable to make it an **environment variable**, which is automatically passed to programs started from that shell.

Assigning Values to Variables

You can **create a shell variable** by using the syntax:
VARIABLE_NAME=value

Viewing All Shell Variables
set
Since the output is very long, it is commonly used with less
set | less

Shell Variables → Used only by **Bash (the shell)**.
Environment Variables → Used by **Bash and all programs launched from Bash**.

What is a Shell Variable ?
NAME=VARIABLE_NAME
echo $NAME
Output =NAME
Only **Bash** knows this variable

What is an Environment Variable?

Suppose you want **other programs** (like Python, Vim, Git, etc.) to also know this variable.
Then export it.

export NAME ->Syntax

Now every program started from this terminal can also access NAME

Why do we need Environment Variables?

Many programs look for certain variables to decide how they should work.

1. HOME

Run : echo $HOME

/home/name ---- Output

When a program wants to save your settings, it asks:

"Where is the user's home folder?"

It checks the **HOME** variable.

What is an Environment Variable?

2. EDITOR

Suppose a program wants to open a text editor.
How does it know which editor you prefer?
RUN : export EDITOR=vim
Now programs that use the EDITOR variable will open **Vim**

3. PATH

Suppose you type
Run : ls
How does Linux know where ls is
It checks the **PATH** variable.
/usr/local/bin Meaning -> This means:Search for commands in these folders.
/usr/local/bin . Where ls finds it runs it

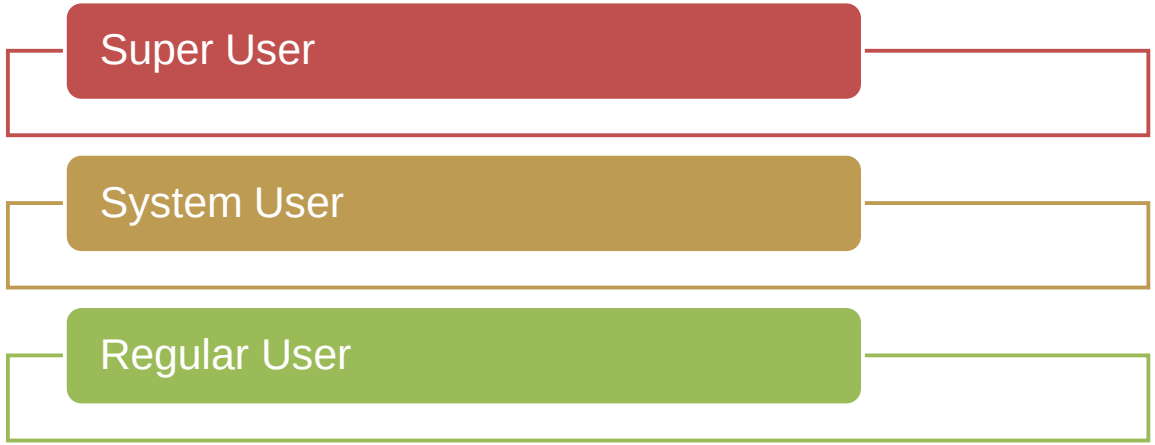
DESCRIBING USER AND GROUP CONCEPTS

WHAT IS A USER?

A user account is used to provide security boundaries between different people and programs that can run commands.

The system distinguishes user accounts by the unique identification number assigned to them, the user ID or UID

Every process (running program) on the system runs as a particular user.



DESCRIBING USER AND GROUP CONCEPTS

```
[user01@host ~]$ id
uid=1000(user01) gid=1000(user01) groups=1000(user01)
```

To view information about the current user that is logged in

```
[user01@host]$ id user02
uid=1002(user02) gid=1001(user02) groups=1001(user02)
```

DESCRIBING USER AND GROUP CONCEPTS

```
[user01@host ~]$ ls -l file1
-rw-rw-r--. 1 user01 user01 0 Feb  5 11:10 file1
[user01@host]$ ls -ld dir1
drwxrwxr-x. 2 user01 user01 6 Feb  5 11:10 dir1
```

To view the owner of a file use the **ls -l** command. To view the owner of a directory use the **ls -** l**d** command

```
[user01@host]$ ps -au
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root       777  0.0  0.0 225752  1496 tty1     Ss+  11:03   0:00 /sbin/agetty -o -
p -- \u --noclear tty1 linux
root       780  0.0  0.1 225392  2064 ttyS0    Ss+  11:03   0:00 /sbin/agetty -o -
p -- \u --keep-baud 115200,38400,9600
user01      1207  0.0  0.2 234044  5104 pts/0    Ss   11:09   0:00 -bash
user01      1319  0.0  0.2 266904  3876 pts/0    R+   11:33   0:00 ps au
```

To view process information, use the **ps** command. The default is to show only processes in the current shell. Add the **a** option to view all processes with a terminal. To view the user associated with a process, include the **u** option

DESCRIBING USER AND GROUP CONCEPTS

```
[user01@host ~]$ ls -l file1
-rw-rw-r--. 1 user01 user01 0 Feb  5 11:10 file1
[user01@host]$ ls -ld dir1
drwxrwxr-x. 2 user01 user01 6 Feb  5 11:10 dir1
```

To view the owner of a file use the **ls -l** command. To view the owner of a directory use the **ls -** l**d** command

```
[user01@host]$ ps -au
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root       777  0.0  0.0 225752  1496 tty1     Ss+  11:03   0:00 /sbin/agetty -o -
p -- \u --noclear tty1 linux
root       780  0.0  0.1 225392  2064 ttyS0    Ss+  11:03   0:00 /sbin/agetty -o -
p -- \u --keep-baud 115200,38400,9600
user01      1207  0.0  0.2 234044  5104 pts/0    Ss   11:09   0:00 -bash
user01      1319  0.0  0.2 266904  3876 pts/0    R+   11:33   0:00 ps au
```

To view process information, use the **ps** command. The default is to show only processes in the current shell. Add the **a** option to view all processes with a terminal. To view the user associated with a process, include the **u** option

CSV 493

LINUX SYSTEM ADMINSTRATION

Unit – II

Controlling Access to Files with Linux File System Permissions : Linux File System Permissions, Managing File System Permissions from the Command Line, Managing Default Permissions and File Access

Monitoring and Managing Linux Processes : Processes, Controlling Jobs, Killing Processes, Monitoring Process Activity

Controlling Services and Daemons : Identifying Automatically Started System Processes, Controlling System Services

Configuring and Securing OpenSSH Service : Accessing the Remote Command Line with SSH, Configuring SSH Key-based Authentication, Customizing SSH Service Configuration

Analyzing and Storing Logs : System Log Architecture, Reviewing Sys log Files, Reviewing systemd Journal Entries, Preserving the systemd Journal, Maintaining Accurate Time

CONTROLLING ACCESS TO FILES

File permissions control access to files. The Linux file permissions system is simple but flexible, which makes it easy to understand and apply, yet still able to handle most normal permission cases easily.

Files have three categories of user to which permissions apply. The file is owned by a user, normally the one who created the file. The file is also owned by a single group, usually the primary group of the user who created the file, but this can be changed.

The most specific permissions take precedence. User permissions override group permissions, which override other permissions

CONTROLLING ACCESS TO FILES

Effects of Permissions on Files and Directories

Permission	Effect on Files	Effect on Directories
r (Read)	Contents of the file can be read.	Contents of the directory (the file names) can be listed.
w (Write)	Contents of the file can be changed.	Files in the directory can be created, deleted, or renamed (provided the user also has execute (x) permission on the directory).
x (Execute)	File can be executed as a command or program.	Directory can be accessed. You can cd into the directory, access its files, and retrieve information about them (subject to the files' own permissions).

CONTROLLING ACCESS TO FILES

VIEWING FILE AND DIRECTORY PERMISSIONS AND OWNERSHIP

```
[user@host~]$ ls -l test
-rw-rw-r--. 1 student student 0 Feb  8 17:36 test
```

The **-l** option of the **ls** command shows more detailed information about file permissions and ownership

```
[user@host ~]$ ls -ld /home
drwxr-xr-x. 5 root root 4096 Jan 31 22:00 /home
```

The first character of the long listing is the file type. You interpret it like this:

- **-** is a regular file.
- **d** is a directory.
- **l** is a soft link.

CONTROLLING ACCESS TO FILES

VIEWING FILE AND DIRECTORY PERMISSIONS AND OWNERSHIP

```
[user@host~]$ ls -l test
-rw-rw-r--. 1 student student 0 Feb  8 17:36 test
```

The **-l** option of the **ls** command shows more detailed information about file permissions and ownership

```
[user@host ~]$ ls -ld /home
drwxr-xr-x. 5 root root 4096 Jan 31 22:00 /home
```

The first character of the long listing is the file type. You interpret it like this:

- **-** is a regular file.
- **d** is a directory.
- **l** is a soft link.

CONTROLLING ACCESS TO FILES

VIEWING FILE AND DIRECTORY PERMISSIONS AND OWNERSHIP

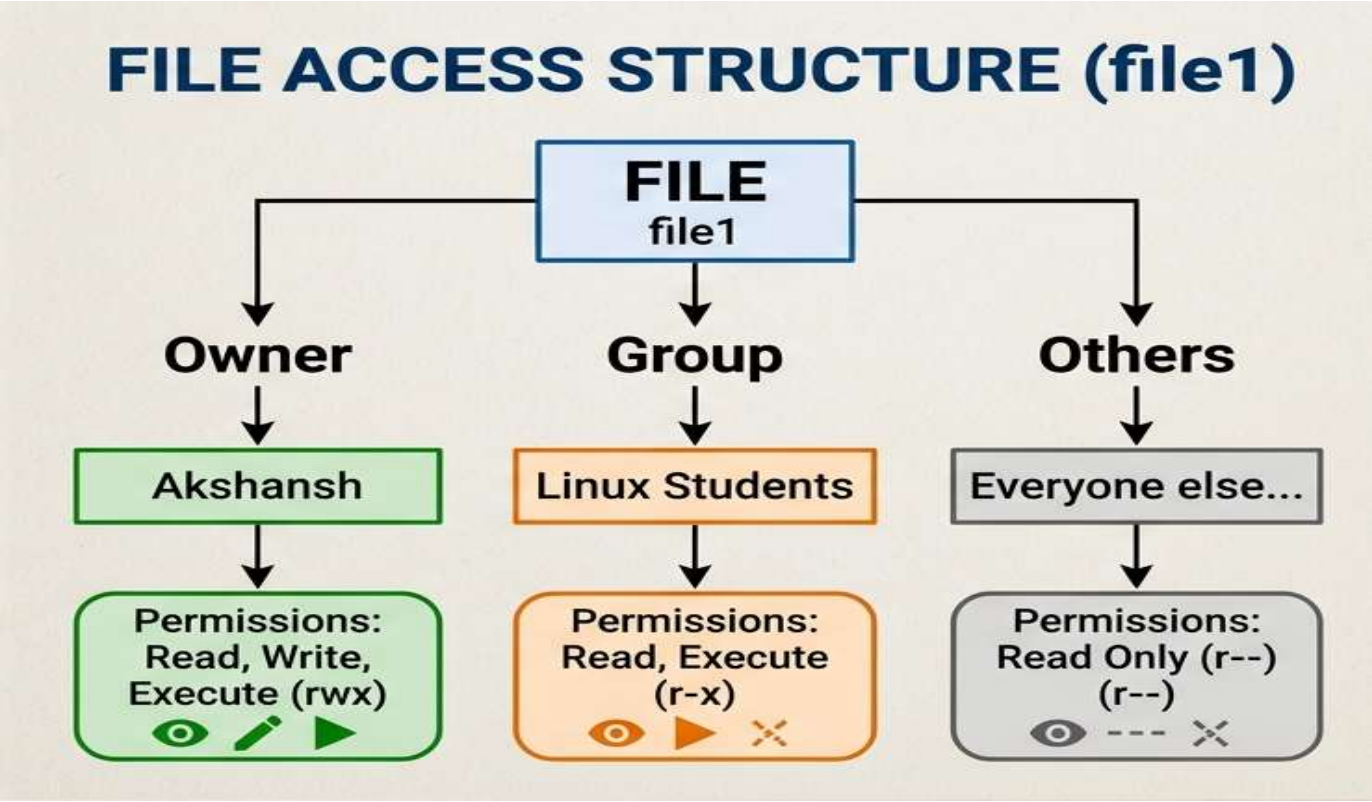
Changing Permissions with the Symbolic Method

```
chmod WhoWhatWhich file|directory
```

- Who is u, g, o, a (for user, group, other, all)
- What is +, -, = (for add, remove, set exactly)
- Which is r, w, x (for read, write, execute)

CONTROLLING ACCESS TO FILES

VIEWING FILE AND DIRECTORY PERMISSIONS AND OWNERSHIP

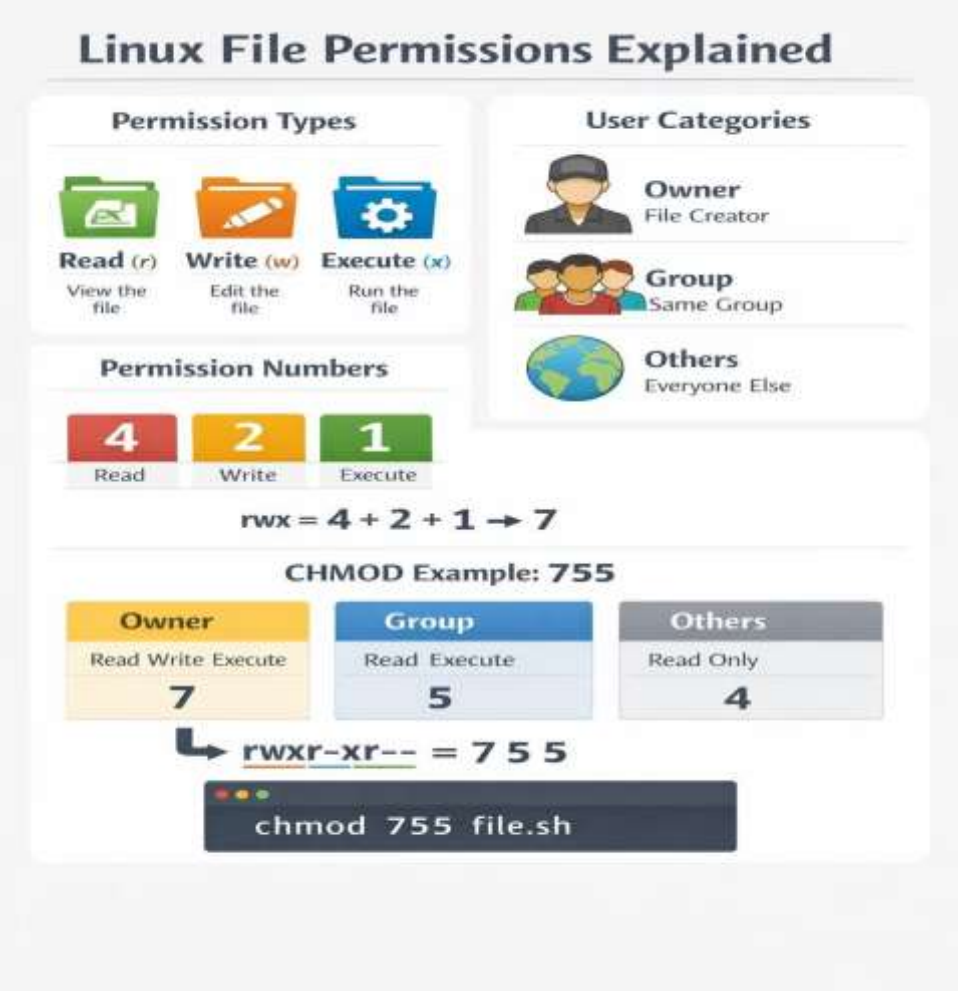


chmod u+x file1
Only **Akshansh** is affected.

chmod g+x file1
Only the **Linux Students** group is affected.

chmod o+x file1
Only everyone else is affected

Managing Files permission Numerically



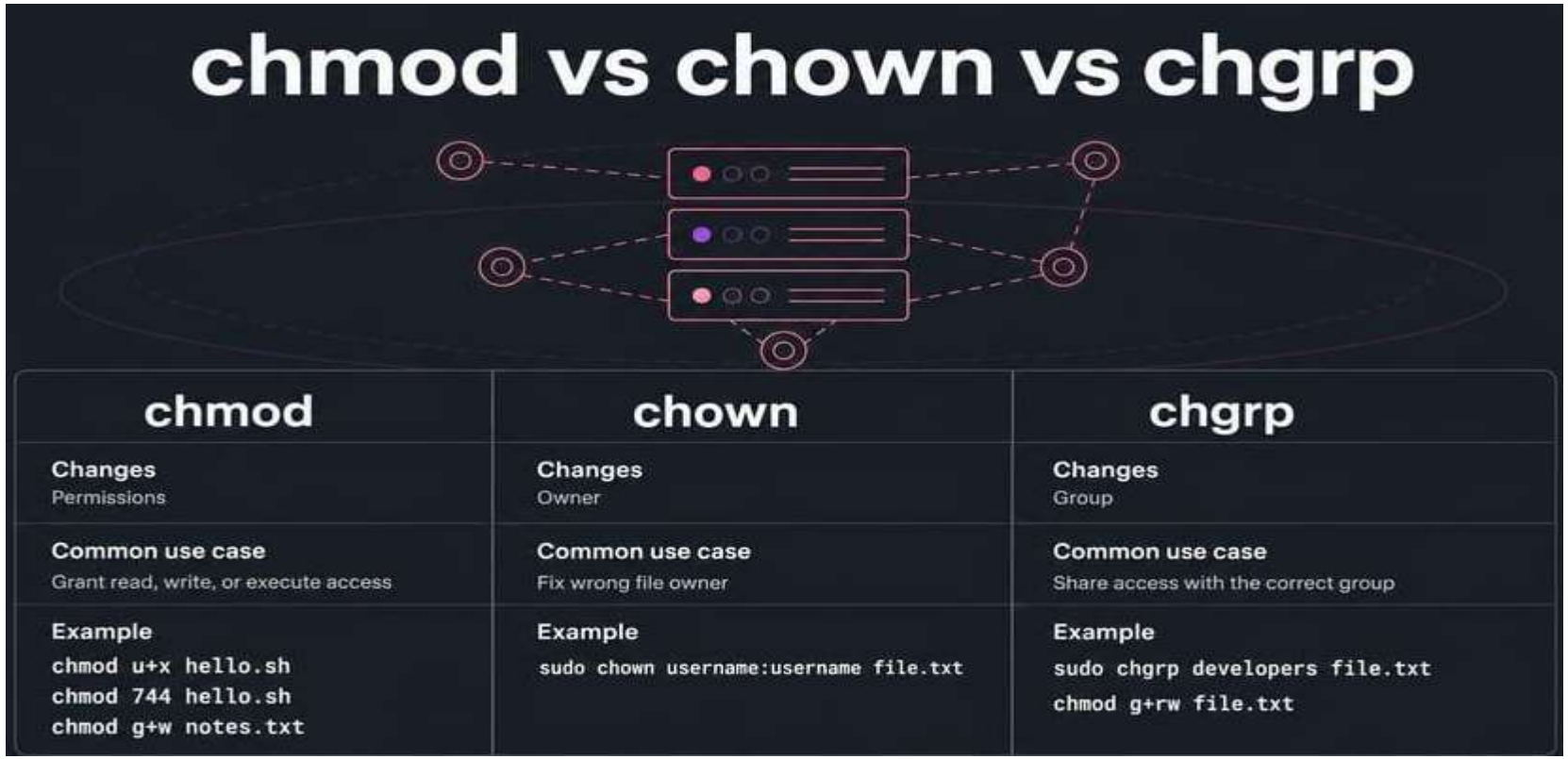
Permission	Calculation	Number
---	0	**0**
--x	1	**1**
-w-	2	**2**
-wx	2 + 1	**3**
r--	4	**4**
r-x	4 + 1	**5**
rw-	4 + 2	**6**
rwx	4 + 2 + 1	**7**

CONTROLLING ACCESS TO FILES

Octal permission mode		
r	w	x
4	2	1
read	write	execute



CONTROLLING ACCESS TO FILES



CONTROLLING ACCESS TO FILES

chown [*user name*] [*file or directory path*]



Recursive — (-R)
change ownership of the directory AND everything inside it.

CONTROLLING ACCESS TO FILES

Cross-Group File Access

There are two teams on the Ubuntu server: A,B

Task 1 — Create Users
Task 2 — Create Groups
Task 3 — Assign Users
Task 4 — Create Project File
Task 5 — Configure Ownership
Task 6 — Restrict the File
Task 7 — Test A ‘s Access
Task 8 — Give A’s Access Through Group Membership

CONTROLLING ACCESS TO FILES

MANAGING DEFAULT PERMISSIONS AND FILE ACCESS

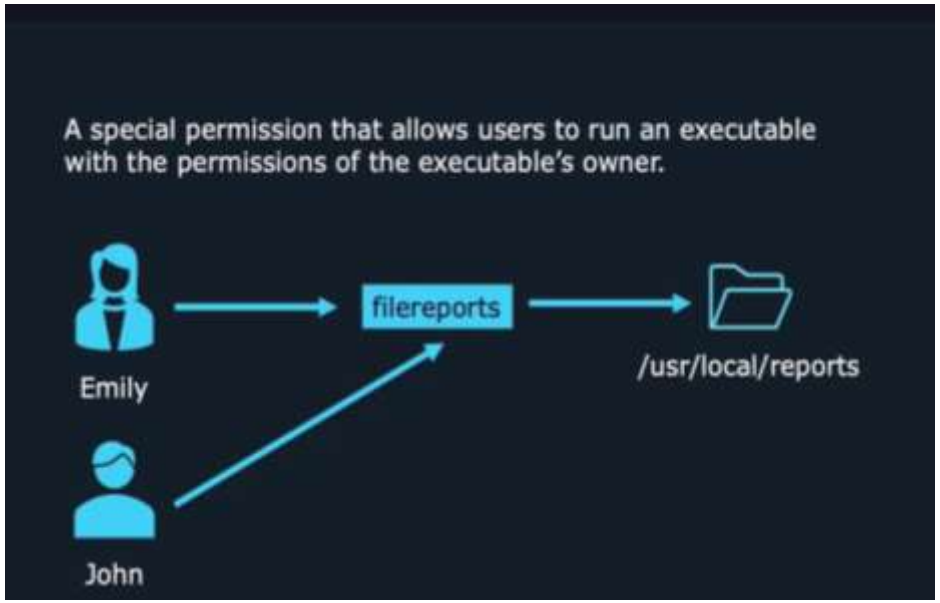
Special permissions constitute a fourth permission type in addition to the basic user, group, and other types. As the name implies, these permissions provide additional access-related features over and above what the basic permission types allow

SPECIAL PERMISSION	EFFECT ON FILES	EFFECT ON DIRECTORIES
u+s (suid)	File executes as the user that owns the file, not the user that ran the file.	No effect.
g+s (sgid)	File executes as the group that owns the file.	Files newly created in the directory have their group owner set to match the group owner of the directory.
o+t (sticky)	No effect.	Users with write access to the directory can only remove files that they own; they cannot remove or force saves to files owned by other users.

CONTROLLING ACCESS TO FILES

MANAGING DEFAULT PERMISSIONS AND FILE ACCESS

SETUID = Run an executable with the permissions of its owner



WITHOUT SETUID

Admin owns the program

change-password

Owner = Admin

Now **User** wants to use it:

User

runs

change-password

Normally, the program runs using **User permissions**.

The problem

User may not have permission to change certain protected information.

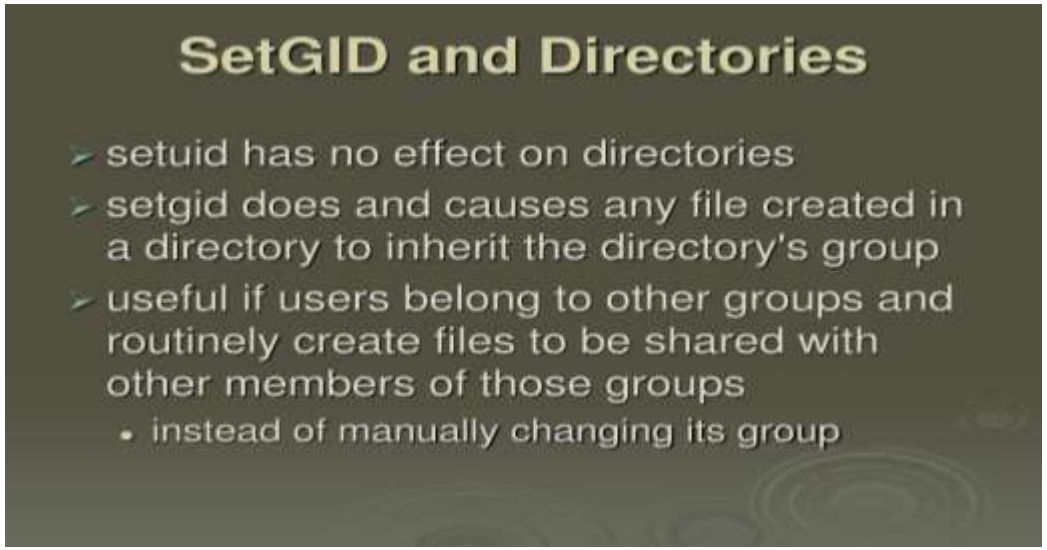
So how can an ordinary user safely use a program that needs special privileges?

SETUID solves this problem.

CONTROLLING ACCESS TO FILES

MANAGING DEFAULT PERMISSIONS AND FILE ACCESS

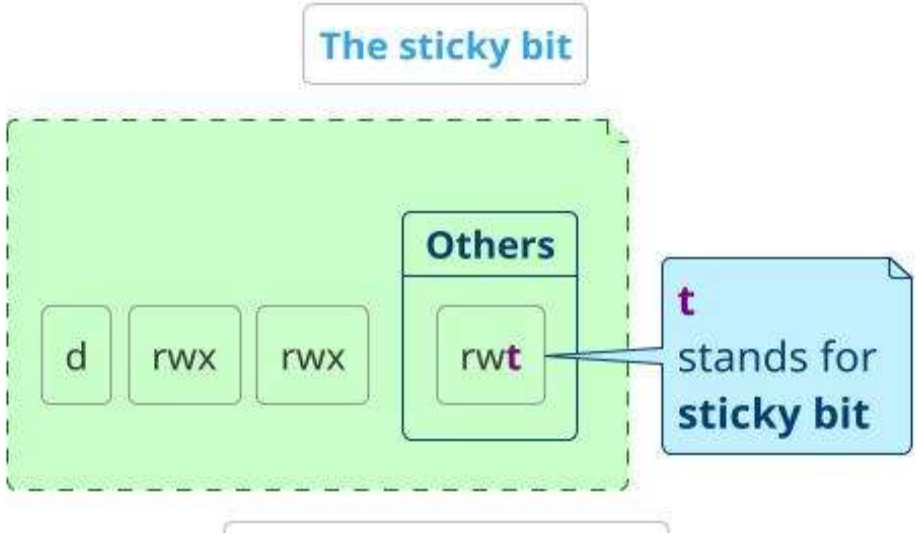
SETGID : "STAY WITH THE GROUP"



Imagine a college project team.**U1,U2,U3**
They all belong to the same team:**AI-Team**
They have a shared folder:
PROJECT/
The folder belongs to:
Group = AI-Team
The problem
U1 creates:
report.txt
Who should the file belong to?
We want:
Owner = U1
Group = AI-Team
Why?Because U2 and U2 are also members of **AI-Team**. **SETGID helps us do this automatically.**

CONTROLLING ACCESS TO FILES

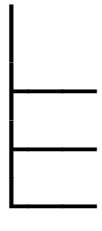
MANAGING DEFAULT PERMISSIONS AND FILE ACCESS



WITHOUT STICKY BIT

Everyone has permission to use the shared folder.ie u1,u2,u3

SHARED/



- u1.txt
- u2.txt
- u3.txt

u3 can access the shared directory.

If the directory permissions allow users to delete entries, u3 may be able to remove:

u1.txt

even though:

Owner = u1

Command : chmod o+t SHARED

MONITORING AND MANAGING LINUX PROCESSES

DEFINITION OF A PROCESS

A process is a running instance of a launched, executable program.

A process consists of:

- An address space of allocated memory
- Security properties including ownership credentials and privileges
- One or more execution threads of program code
- Process state

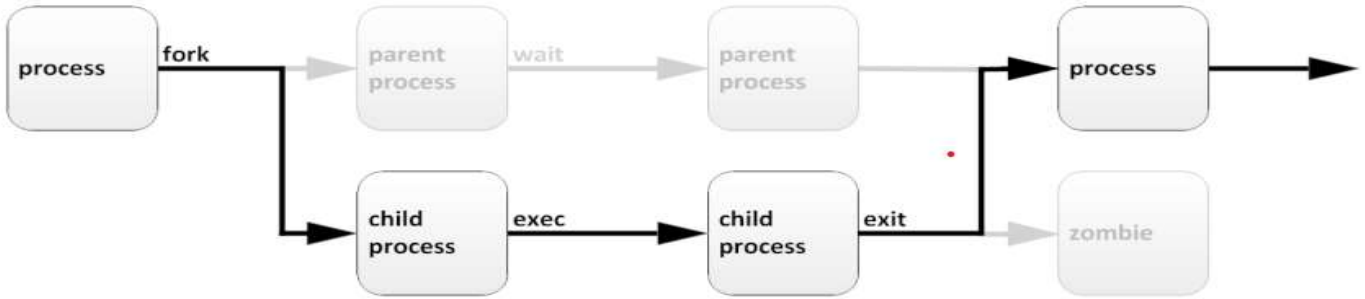
The environment of a process includes:

- Local and global variables
- A current scheduling context
- Allocated system resources, such as file descriptors and network ports

MONITORING AND MANAGING LINUX PROCESSES

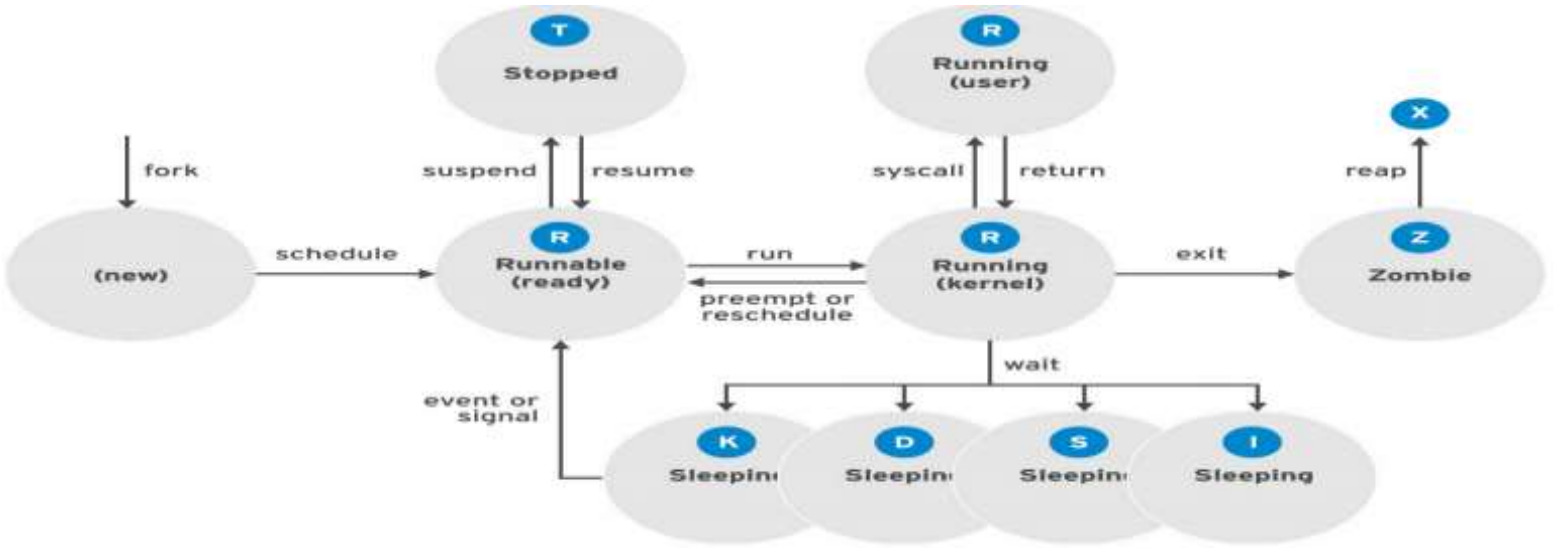
An existing (parent) process duplicates its own address space (**fork**) to create a new (child) process structure. Every new process is assigned a unique process ID (PID) for tracking and security. The PID and the parent's process ID (PPID) are elements of the new process environment. Any process may create a child process.

Through the fork routine, a child process inherits security identities, previous and current file descriptors, port and resource privileges, environment variables, and program code. A child process may then exec its own program code. Normally, a parent process sleeps while the child process runs, setting a request (wait) to be signalled when the child completes. Upon exit, the child process has already closed or discarded its resources and environment. The only remaining resource, called a zombie, is an entry in the process table



MONITORING AND MANAGING LINUX PROCESSES

In a multitasking operating system, each CPU (or CPU core) can be working on one process at a single point in time. As a process runs, its immediate requirements for CPU time and resource allocation change. Processes are assigned a state, which changes as circumstances dictate.



A **zombie process** is a terminated child process that remains in the system's process table because its parent process has not yet read its exit status. It uses no CPU or memory, but holds a Process ID (PID) slot

NAME	FLAG	KERNEL-DEFINED STATE NAME AND DESCRIPTION
Running	**R**	TASK_RUNNING: The process is either executing on a CPU or waiting to run. Process can be executing user routines or kernel routines (system calls), or be queued and ready when in the *Running* (or *Runnable*) state.
Sleeping	**S**	TASK_INTERRUPTIBLE: The process is waiting for some condition: a hardware request, system resource access, or signal. When an event or signal satisfies the condition, the process returns to *Running*.
	D	TASK_UNINTERRUPTIBLE: This process is also *Sleeping*, but unlike **S** state, does not respond to signals. Used only when process interruption may cause an unpredictable device state.
	K	TASK_KILLABLE: Identical to the uninterruptible **D** state, but modified to allow a waiting task to respond to the signal that it should be killed (exit completely). Utilities frequently display *Killable* processes as **D** state.

NAME	FLAG	KERNEL-DEFINED STATE NAME AND DESCRIPTION
	I	TASK_REPORT_IDLE: A subset of state **D**. The kernel does not count these processes when calculating load average. Used for kernel threads. Flags TASK_UNINTERRUPTABLE and TASK_NOLOAD are set. Similar to TASK_KILLABLE, also a subset of state **D**. It accepts fatal signals.
Stopped	**T**	TASK_STOPPED: The process has been *Stopped* (suspended), usually by being signaled by a user or another process. The process can be continued (resumed) by another signal to return to *Running*.
	T	TASK_TRACED: A process that is being debugged is also temporarily *Stopped* and shares the same **T** state flag.
Zombie	**Z**	EXIT_ZOMBIE: A child process signals its parent as it exits. All resources except for the process identity (PID) are released.
	X	EXIT_DEAD: When the parent cleans up (*reaps*) the remaining child process structure, the process is now released completely. This state will never be observed in process-listing utilities.

Why Process States are Important

When troubleshooting a system, it is important to understand how the kernel communicates with processes and how processes communicate with each other. At process creation, the system assigns the process a state. The **S** column of the **top** command or the **STAT** column of the **ps** show the state of each process. On a single CPU system, only one process can run at a time. It is possible to see several processes with a state of **R**. However, not all of them will be running consecutively, some of them will be in status waiting.

```
[user@host ~]$ top
PID USER   PR   NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  1 root   20    0  244344  13684   9024 S   0.0   0.7   0:02.46 systemd
  2 root   20    0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
...output omitted...
```

The top command is a real-time system monitoring utility available on Linux systems. It presents a continuously updated view of running processes along with overall system resource usage such as CPU load, memory consumption, and system uptime

LISTING PROCESSES

The **ps** command is used for listing current processes. It can provide detailed process information, including:

- User identification (UID), which determines process privileges
- Unique process identification (PID)
- CPU and real time already expended
- How much memory the process has allocated in various locations
- The location of process **stdout**, known as the controlling terminal
- The current process state.

The ps aux command in Linux is used to display a **comprehensive, static snapshot of all currently running processes** on the system. It is one of the most widely used tools by system administrators for monitoring resource usage, troubleshooting system health, and identifying process

```
[user@host ~]$ ps aux
USER       PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.1  0.1  51648  7504 ?        Ss   17:45   0:03 /usr/lib/systemd/
syst
root         2  0.0  0.0      0     0 ?        S    17:45   0:00 [kthreadd]
root         3  0.0  0.0      0     0 ?        S    17:45   0:00 [ksoftirqd/0]
root         5  0.0  0.0      0     0 ?        S<   17:45   0:00 [kworker/0:0H]
root         7  0.0  0.0      0     0 ?        S    17:45   0:00 [migration/0]
```

Controlling Jobs

DESCRIBING JOBS AND SESSIONS

Job control is a feature of the shell which allows a single shell instance to run and manage multiple commands.

Types of Jobs :
A **foreground job** is a job that is **currently running in the terminal and has control of the keyboard**
Foreground = The job currently using the terminal.

A **background job** is a job that **runs without controlling the keyboard**, allowing you to continue using the terminal for other commands

Examples
Foreground Job Example: Running sleep 100 or sudo apt update. The terminal stays busy, shows the progress or waits, and does not let you type a new command until it is done or stopped
Background Job Example: Running sleep 500 &. Adding the & symbol tells the system to run the sleep timer in the background, immediately returning your command prompt so you can do other work.

Controlling Jobs

Cron Job: Cron is a time-based job scheduling utility available in Linux and Unix-like operating systems. It runs as a background daemon process and automatically executes scheduled tasks at predefined times without user intervention

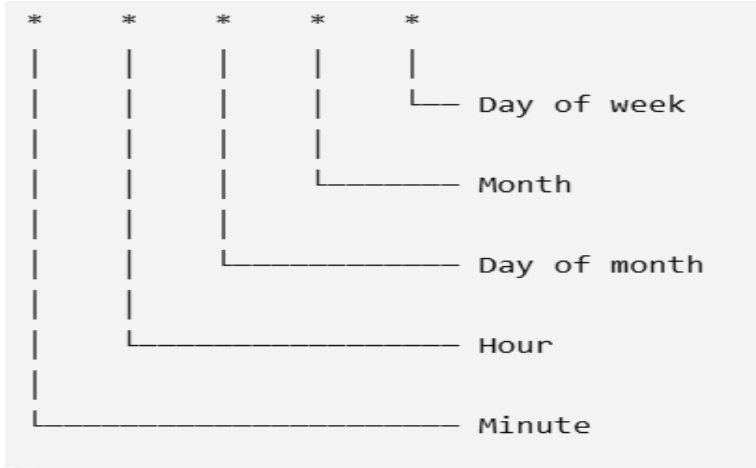
Working of Cron :

Cron operates as a background service that continuously monitors scheduled jobs and executes them when their defined time matches the system time

	Command	Purpose	Example
1	crontab -e	**Create or edit** your cron jobs	crontab -e
2	crontab -l	**List/view** your existing cron jobs	crontab -l
3	crontab -r	**Remove all** cron jobs for the current user	crontab -r
4	crontab -i	**Remove all** cron jobs with confirmation	crontab -i
5	crontab -u username -e	**Edit another user's** crontab (requires appropriate privileges)	sudo crontab -username -e

Creating a Cron Job demonstration :

Step 1 — Create a new crontab
crontab –e
Your editor will open.At the bottom, add **only this line**:
* * * * * echo "Cron is working" >> /tmp/cron_test.txt



Step 2 - Verify the job exists
crontab –l

Step 3 - Wait 1–2 minutes
Then run : cat /tmp/cron_test.txt
cron's basic job is simply:At a specified time, automatically execute this command.

Step 4 crontab –r
To remove job

KILLING PROCESSES

PROCESS CONTROL USING SIGNALS

A signal is a software interrupt delivered to a process. Each signal has a default action, usually one of the following:

- Term - Cause a program to terminate (exit) at once.
- Core - Cause a program to save a memory image (core dump), then terminate.
- Stop - Cause a program to stop executing (suspend) and wait to continue (resume).

SIGNAL NUMBER	SHORT NAME	DEFINITION	PURPOSE
1	HUP	Hangup	Used to report termination of the controlling process of a terminal. Also used to request process reinitialization (configuration reload) without termination.
2	INT	Keyboard interrupt	Causes program termination. Can be blocked or handled. Sent by pressing INTR key combination (**Ctrl+C**).
3	QUIT	Keyboard quit	Similar to SIGINT, but also produces a process dump at termination. Sent by pressing QUIT key combination (**Ctrl+**).
9	KILL	Kill, unblockable	Causes abrupt program termination. Cannot be blocked, ignored, or handled; always fatal.
15 *default*	TERM	Terminate	Causes program termination. Unlike SIGKILL, can be blocked, ignored, or handled. The "polite" way to ask a program to terminate; allows self-cleanup.
18	CONT	Continue	Sent to a process to resume, if stopped. Cannot be blocked. Even if handled, always resumes the process.

KILLING PROCESSES

The **kill** command sends a signal to a process by PID number. Despite its name, the kill command can be used for sending any signal, not just those for terminating programs. You can use the **kill -l** command to list the names and numbers of all available signals

```
[user@host ~]$ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
...output omitted...
[user@host ~]$ ps aux | grep job
5194  0.0  0.1 222448  2980 pts/1    S    16:39   0:00 /bin/bash /home/user/bin/
control job1
5199  0.0  0.1 222448  3132 pts/1    S    16:39   0:00 /bin/bash /home/user/bin/
control job2
5205  0.0  0.1 222448  3124 pts/1    S    16:39   0:00 /bin/bash /home/user/bin/
control job3
5430  0.0  0.0 221860  1096 pts/1    S+   16:41   0:00 grep --color=auto job
```

KILLING PROCESSES

The **kill** command sends a signal to a process by PID number. Despite its name, the kill command can be used for sending any signal, not just those for terminating programs. You can use the **kill -l** command to list the names and numbers of all available signals

```
[user@host ~]$ kill 5194
[user@host ~]$ ps aux | grep job
user    5199  0.0  0.1 222448  3132 pts/1    S    16:39   0:00 /bin/bash /home/
user/bin/control job2
user    5205  0.0  0.1 222448  3124 pts/1    S    16:39   0:00 /bin/bash /home/
user/bin/control job3
user    5783  0.0  0.0 221860   964 pts/1    S+   16:43   0:00 grep --color=auto
 job
[1]   Terminated              control job1
[user@host ~]$ kill -9 5199
[user@host ~]$ ps aux | grep job
user    5205  0.0  0.1 222448  3124 pts/1    S    16:39   0:00 /bin/bash /home/
user/bin/control job3
user    5930  0.0  0.0 221860  1048 pts/1    S+   16:44   0:00 grep --color=auto
 job
[2]-  Killed                  control job2
[user@host ~]$ kill -SIGTERM 5205
user    5986  0.0  0.0 221860  1048 pts/1    S+   16:45   0:00 grep --color=auto
 job
[3]+  Terminated              control job3
```

RHEL System Administration – Scenario-Based Test

Scenario:
You are a Linux System Administrator for a college laboratory. A RHEL server is used for development and student data storage. During routine maintenance, you must configure the network, manage software, organize files, and prepare a separate filesystem for student data.

1. Network Configuration
Task: Configure 192.168.10.50/24, gateway 192.168.10.1 and DNS 8.8.8.8 using nmcli. Activate the connection.

2. Network Validation
Task: Verify the server's IP address, routing information and network connectivity using suitable commands.

3. Hostname
Task: Change localhost.localdomain to server01.lab.local and verify the new hostname.

4. Locate a File
Task: Locate lab.conf somewhere under /etc using an appropriate Linux command.

5. Compressed Archive
Task: Create project_backup.tar.gz from /home/student/project and verify its contents without extracting it.

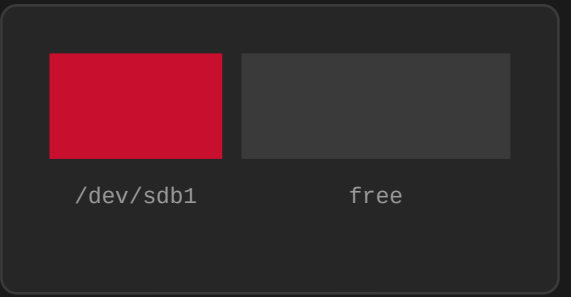
6. Mount Filesystem
Task: For XFS partition /dev/sdb1, create /data, mount it there and verify that it is accessible.

7. Unmount Filesystem
Task: Verify whether /data is mounted and safely unmount the filesystem.

8. Symbolic Link
Task: Create /home/student/report.txt as a symbolic link to /data/project/report.txt and verify it.

9. Software Package
Task: Search for httpd, install it if required, and verify the installation and package information using RPM.

10. Software Updates
Task: Display enabled repositories, check available updates and update installed packages using Yum/DNF.



RED HAT ENTERPRISE LINUX

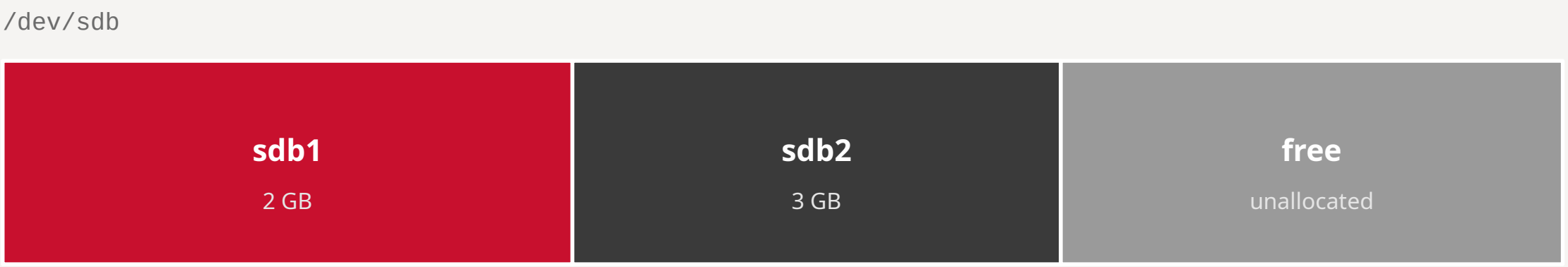
Disk Partitioning & Mounting

Partition → Format → Mount → Persist

9 SLIDES · SYSTEM ADMINISTRATION

What Is a Partition?

A logical slice of a physical disk



One physical disk can hold multiple partitions — each behaves like its own independent storage device.

MBR

up to 4 primary partitions

GPT

up to 128 partitions

The Tools You'll Use

Three commands cover the whole workflow

1

`lsblk`

See disks & partitions

2

`fdisk / parted`

Create & edit partitions

3

`mkfs`

Format with a filesystem

Create a Partition

Four keystrokes inside fdisk

```
sudo fdisk /dev/sdb
```

1

```
n
```

New partition

2

```
p
```

Primary type

3

```
+2G
```

Set size

4

```
w
```

Write & exit

Then format & verify:

```
lsblk
```

Format the Partition

Give it a filesystem before it can store files

XFS

```
mkfs.xfs /dev/sdb1
```

RHEL default

ext4

```
mkfs.ext4 /dev/sdb1
```

Widely compatible

vfat

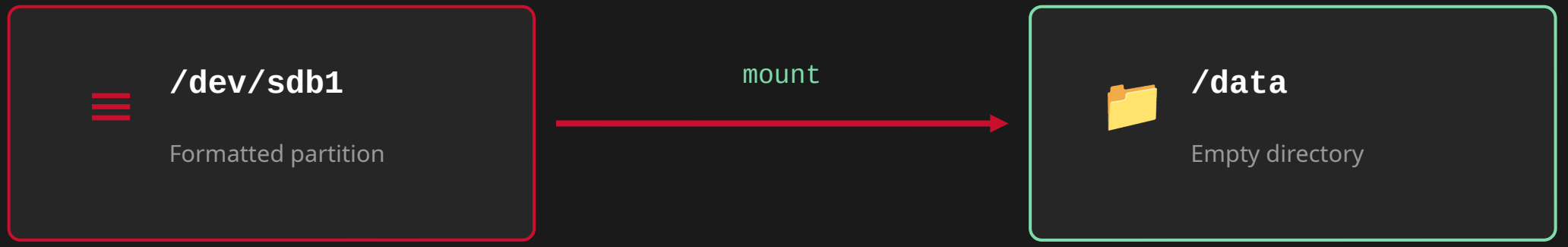
```
mkfs.vfat /dev/sdb1
```

Cross-platform

Pick one filesystem per partition — formatting erases existing data.

What Is a Mount Point?

A directory that gives the partition an address in the filesystem tree



Files written to /data now live on /dev/sdb1.

```
sudo mkdir /data
```

Mount It

Attach the partition to its mount point — active until reboot

```
sudo mount /dev/sdb1 /data
```

```
df -h
```

confirm it's mounted

```
mount | grep sdb1
```

see mount options

```
lsblk
```

view the mountpoint column

A plain mount is temporary — it disappears on reboot.

Make the Mount Persistent

Add an entry to /etc/fstab so it survives reboot

```
blkid /dev/sdb1
```

get its UUID

```
sudo nano /etc/fstab

UUID=xxxx-xxxx  /data  xfs  defaults  0 0
```

device · mountpoint · fstype · options · dump · fsck

✓ **Test before rebooting**

```
sudo mount -a
```

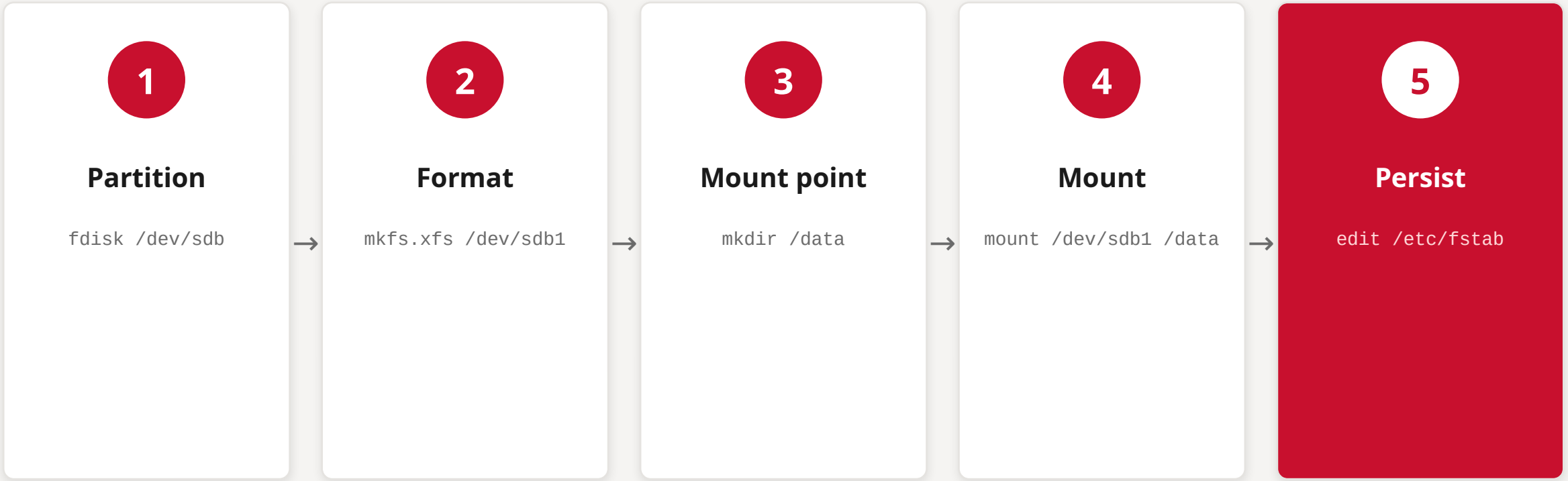
Re-mounts everything in fstab — catches typos immediately instead of at next boot.

No errors + df -h shows /data?

You're done — the mount survives every future reboot.

Recap

The full path from raw disk to persistent storage



RHEL Shell Scripting – Scenario-Based Questions

Q1. Student Name

Write a Bash script to accept a student's name and display: Welcome, <name>.

Q2. Student Marks

Accept marks of a student and display the entered marks.

Q3. Total Marks

Accept marks in 3 subjects and calculate the total marks.

Q4. Average & Result

Calculate average marks and display Pass if average ≥ 40, otherwise Fail.

Q5. Highest Marks

Accept marks of 3 students and display the student with the highest marks.

Q6. Grade

Accept marks and display grade: A (≥80), B (≥60), C (≥50), D (≥40), otherwise F.

Q7. Student Details

Using positional arguments, accept Name, Roll No., Course, Semester and Department and display them.

Q8. Student File

Using positional arguments, accept source file and destination directory, then copy the student record file.

Q9. Multiple Students

Use a loop to accept details of 5 students and display each student's name and marks.

Q10. Final Task

Create one script using positional arguments + condition + loop to process student records and copy the final report to a destination.

Instruction: Write, save, execute and demonstrate each Bash script on RHEL/Linux.

Running Loops and Conditional Commands

Bash control structures: for, while, until, if / elif / else

```
for HOST in host1 host2; do echo $HOST; done
```

What You Will Learn

Loops

Repeat a command with for, while, until

Exit codes

Know if a command succeeded (0) or failed

Tests

Compare numbers, strings and files

Decisions

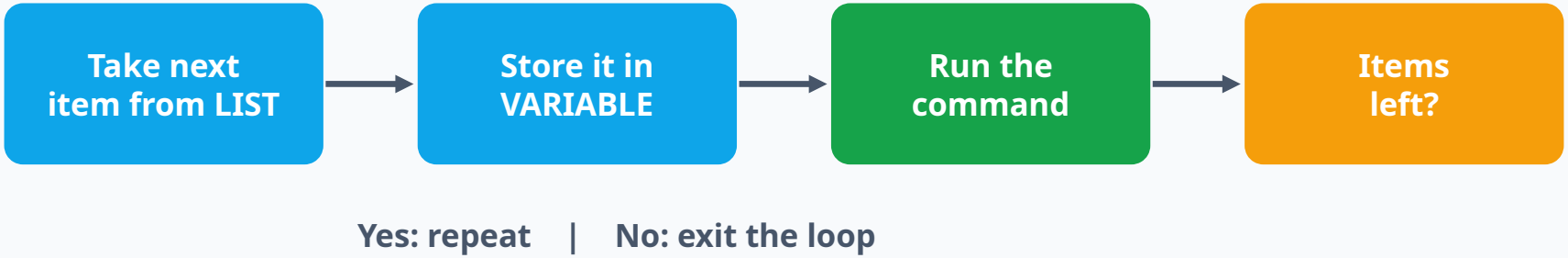
Run different tasks with if / elif / else

The for Loop

Repeat a task once per item in a list

```
for VARIABLE in LIST; do
    COMMAND $VARIABLE
done
```

Use it to check many targets: back up many databases, ping many hosts.



Where Does the LIST Come From?

Typed list

Command

```
for H in host1 host2 host3
```

Items processed

```
host1 host2 host3
```

Brace expansion

Command

```
for H in host{1..3}
for F in file{a..c}
```

Items processed

```
host1 host2 host3
filea fileb filec
```

Command output

Command

```
for N in $(seq 2 2 10)
```

Items processed

```
2 4 6 8 10
```

for Loop in Action

Print even numbers from 1 to 10

```
for EVEN in $(seq 2 2 10)
do echo "$EVEN"
done
```

seq FIRST STEP LAST makes the list.
Each number goes into $EVEN, one at a time.

Output

```
2
4
6
8
10
```

Tip: the variable name is your choice.

Exit Codes

Every command reports how it finished (0 to 255)

0
Success

1 - 255
Error

Read the last exit code with

```
echo $?
```

Leave a script early with exit N (no N = last command's code)

Exit Codes: true and false

```
$ /bin/true
$ echo $?
0
```

Always succeeds

```
$ /bin/false
$ echo $?
1
```

Always fails

Script example: #!/usr/bin/bash -> echo "Hello, world" -> exit 0

Testing a Condition

True = exit code 0. False = non-zero.

test command

```
test 1 -gt 0 ; echo $?
0
test 0 -gt 1 ; echo $?
1
```

[[...]] syntax

```
[[ 1 -eq 1 ]]; echo $?
0
[[ 1 -ne 1 ]]; echo $?
1
```

Spaces inside the brackets are mandatory: [[$x -gt 1]]

Numeric Operators

Operator	Meaning	Example	Result
-eq	equal	[[1 -eq 1]]	0 (true)
-ne	not equal	[[1 -ne 1]]	1 (false)
-gt	greater than	[[8 -gt 2]]	0 (true)
-ge	greater or equal	[[2 -ge 2]]	0 (true)
-lt	less than	[[2 -lt 2]]	1 (false)
-le	less or equal	[[3 -le 5]]	0 (true)

String and File Operators

String	Meaning
[[abc = abc]]	equal
[[abc != def]]	not equal
[[-z "$S"]]	string is empty
[[-n "$S"]]	string is not empty

File	Meaning
[[-f FILE]]	regular file exists
[[-d DIR]]	directory exists
[[-r FILE]]	you can read it

More operators: man test

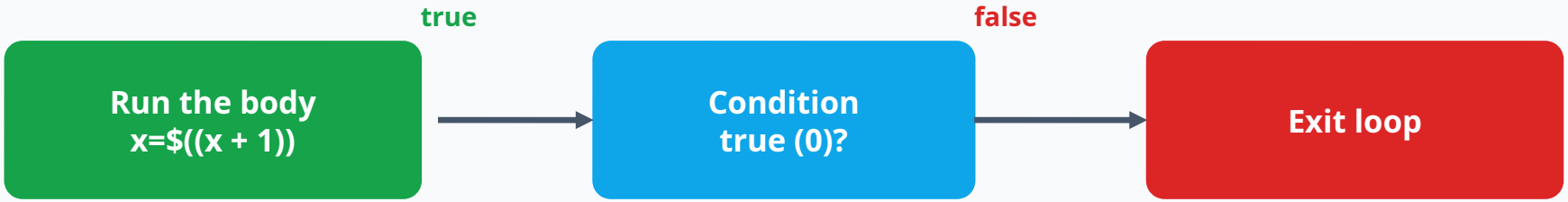
The while Loop

Repeat while the condition is TRUE

```
while <CONDITION>; do
    <STATEMENT>
done
```

```
x=1
while [[ $x -le 5 ]]; do
  echo "x is $x"
  x=$((x + 1))
done
```

```
x is 1
x is 2
x is 3
x is 4
x is 5
```



After the body runs, the condition is checked again

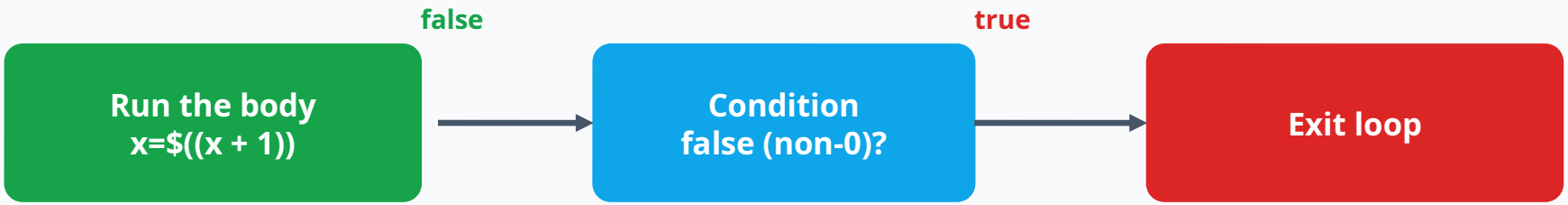
The until Loop

Repeat until the condition becomes TRUE

```
until <CONDITION>; do
    <STATEMENT>
done
```

```
x=5
until [[ $x -lt 1 ]]; do
  echo "x is $x"
  x=$((x - 1))
done
```

```
x is 5
x is 4
x is 3
x is 2
x is 1
```



After the body runs, the condition is checked again

while vs until

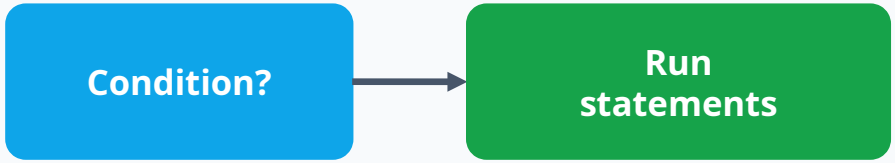
while	until
Runs while the condition is TRUE (exit code 0) Stops when it turns false	Runs while the condition is FALSE (non-zero) Stops when it turns true

Both test the condition BEFORE the first pass. If it blocks the first pass, the body never runs.

if / then

Do something only when a condition is true

```
if <CONDITION>; then
    <STATEMENT>
fi
```



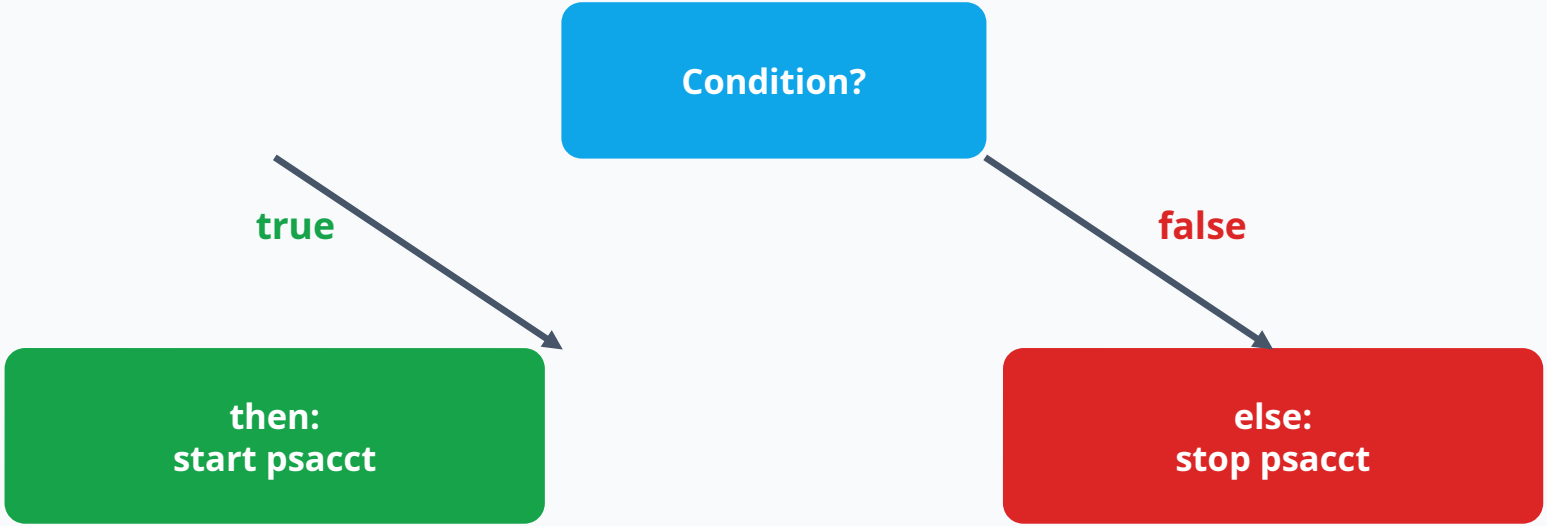
false: skip, nothing happens

Example: start psacct if it is not running

```
systemctl is-active psacct > /dev/null 2>&1
if [[ $? -ne 0 ]]; then sudo systemctl start psacct; fi
```

if / then / else

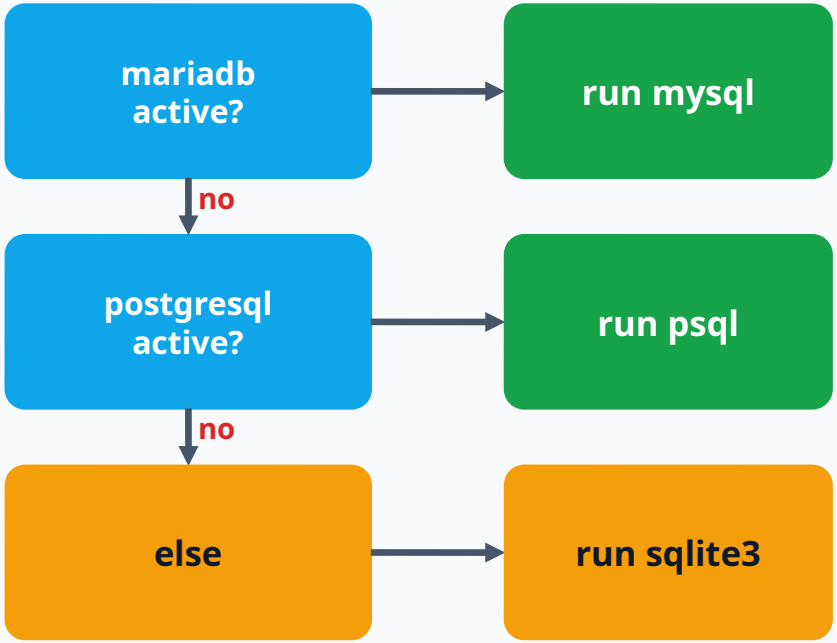
Pick one of two paths




```
if [[ $? -ne 0 ]]; then
    sudo systemctl start psacct
else
    sudo systemctl stop psacct
fi
```

if / elif / else

Test several conditions in order; first true wins




```
if [[ "$MARIADB_ACTIVE" -eq 0 ]]; then
    mysql
elif [[ "$POSTGRESQL_ACTIVE" -eq 0 ]]; then
    psql
else
    sqlite3
fi
```

Quick Reference

Construct	Use it when	Keywords
for	Run once per item in a list	for … in …; do … done
while	Repeat while a condition is true	while …; do … done
until	Repeat until a condition is true	until …; do … done
if / then	Act only if true	if …; then … fi
if / else	Choose between 2 paths	… else … fi
elif	Choose among many paths	… elif …; then …
$?	Check last exit code (0 = OK)	echo $?

LINUX SYSTEM ADMINISTRATION • GUIDED EXERCISE

Regular Expressions & Pattern Matching in Linux

Searching • Filtering • Validating • Extracting text

```
$ grep -E '^A' students.txt
Amit CSE
```

^ $. * + ?

[ ] { } |

TOOLS

grep **grep -E** **sed** **awk**

What Is Pattern Matching?

Pattern matching means searching text for characters or sequences that satisfy a specified pattern.

EXAMPLE FILE: students.txt

Amit CSE
Rahul CSE
Neha ECE
Priya CSE
Rohan ME

COMMAND

```
$ grep "CSE" students.txt
```

OUTPUT

```
Amit CSE
Rahul CSE
Priya CSE
```

3 of 5 lines contain “CSE”

What Is a Regular Expression?

A regular expression (regex) is a special pattern used to describe text.

SIMPLE SEARCH

grep "CSE" students.txt

REGEX SEARCH

grep -E "^A" students.txt

^A means: lines that start with A.

COMMAND

```
$ grep -E "^A" students.txt
```

OUTPUT

```
Amit CSE
```

PATTERN BREAKDOWN

^	start of line
A	the letter A

CORE SYMBOLS

Basic Regex Symbols

^ Start of line	$ End of line	. Any single character	[abc] a, b, or c
[^0-9] Not a digit	[0-9] Any digit	+ One or more	* Zero or more
? Zero or one	{n} Exactly n	{n,m} n to m occurrences	| OR

^ and $ — Start and End of Line

^

^app → line must begin with app

COMMAND

```
$ grep '^app' fruits.txt
```

OUTPUT

```
apple
application
```

$

$ → line must end with the specified pattern

COMMAND

```
$ grep 'CSE$' students.txt
```

OUTPUT

```
Amit CSE
Rahul CSE
Priya CSE
```

. and [ ] — Character Matching

. = any one character

COMMAND

```
$ grep -E 'c.t' file.txt
```

OUTPUT

```
cat
cut
cot
c9t
```

[ ] [0-9] = any digit from 0 to 9

COMMAND

```
$ grep -E '[0-9]' file.txt
```

CHARACTER SETS

[0-9] digit
[a-z] lowercase letter
[A-Z] uppercase letter
[^0-9] not a digit

Quantifiers: *, + and ?

*

ab*

a followed by zero or more b's

+

ab+

a followed by one or more b's

?

colou?r

u is optional

COMMAND

```
$ grep -E 'ab+' file.txt
```

OUTPUT

```
ab
abb
abbb
```

Use grep -E for extended operators such as + and ?.

{n}, {n,} and {n,m} — Repetition

{n}

^[0-9]{5}$

exactly 5 digits

{n,}

^[0-9]{3,}$

3 or more digits

{n,m}

^[0-9]{3,5}$

3 to 5 digits

COMMAND

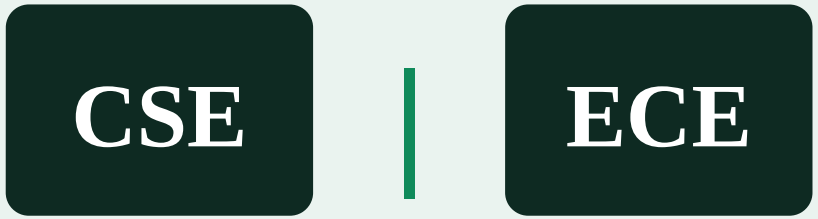
```
$ grep -E '^[0-9]{5}$' data.txt
```

OUTPUT

```
12345
98765
```

^ and $ anchor the pattern, so the whole line must match.

| — OR Operator



CSE|ECE means: CSE OR ECE

Useful for filtering multiple alternatives.

COMMAND

```
$ grep -E 'CSE|ECE' students.txt
```

OUTPUT

```
Amit CSE
Rahul CSE
Neha ECE
Priya CSE
```

APPLICATIONS

Email Validation with Regex

USERNAME	@	DOMAIN	.	EXTENSION
^[A-Za-z0-9._%+-]+	@	[A-Za-z0-9.-]+	\.	[A-Za-z]{2,}$

COMMAND

```
$ grep -E '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$' emails.txt
```

OUTPUT

```
student@gmail.com
teacher@lpu.in
admin@company.org
john.doe@gmail.com
```

Regex in a Shell Script

SCRIPT

```
#!/bin/bash
echo "Enter email:"
read email

if echo "$email" | grep -Eq '^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}$'
then
    echo "Valid email"
else
    echo "Invalid email"
fi
```

RUN 1

```
Enter email:
student@gmail.com
Valid email
```

RUN 2

```
Enter email:
student@gmail
Invalid email
```

grep vs grep -E

grep

BRE

- Basic Regular Expressions (BRE)
- Good for simple searches and basic patterns

EXAMPLE

```
$ grep "error" logfile.txt
```

grep -E

ERE

- Extended Regular Expressions (ERE)
- Easier use of +, ?, |, {n,m}

EXAMPLE

```
$ grep -E 'ERROR|WARNING' logfile.txt
```

APPLICATIONS

Why Regex Is Important in Linux

01

Search text in files

02

Filter command output

03

Validate email, phone, username, etc.

04

Analyze system/application logs

05

Extract useful information

06

Build decision-making shell scripts

COMMON TOOLS **grep • sed • awk**

PRACTICE

Practice Tasks

1. Find lines beginning with a vowel.
2. Find lines ending with CSE.
3. Find names containing exactly five characters.
4. Find numbers containing exactly four digits.
5. Find numbers beginning with 9.
6. Find lines containing CSE or ECE.
7. Validate a phone number using regex.
8. Validate an email using regex.
9. Check a username containing only letters and digits.
10. Write a shell script for email validation.

PRACTICE

Quick Revision

`^abc`	starts with abc	`[0-9]{5}`	exactly 5 digits
`abc$`	ends with abc	`a+`	one or more a
`[0-9]`	digit	`a*`	zero or more a
`[A-Z]`	uppercase letter	`a?`	zero or one a
`[a-z]`	lowercase letter	`cat|dog`	cat OR dog

Remember: Regex describes the pattern; grep applies that pattern to text.

Regular Expressions

Matching Text in RHEL / Linux

Simple explanation + practical grep examples

```
grep
^ERROR
error$
[0-9]
.*
```

Regex = a pattern used to find text

What is a Regular Expression?

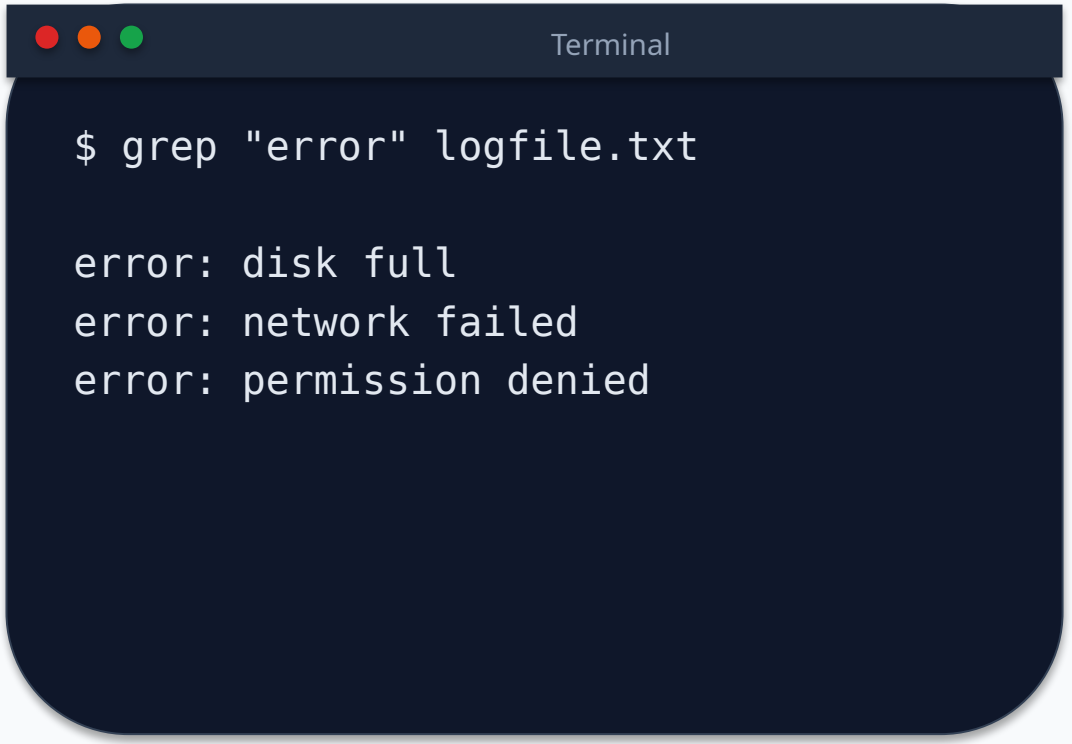
Think of regex as a smart search pattern.

A regular expression (regex) is a pattern used to find specific text.

It is more powerful than a simple word search.

In RHEL/Linux, grep is one of the most useful commands for regex.

Regex can also be used with tools such as less and vim.



Search pattern → Matching lines

Why Do We Need Regex?

Useful when files or logs contain thousands of lines.

Terminal

```
server started
user admin logged in
ERROR: disk full
user john logged in
ERROR: network failed
backup completed
ERROR: permission denied
```

We want only ERROR lines

Terminal

```
$ grep "ERROR" logfile.txt

ERROR: disk full
ERROR: network failed
ERROR: permission denied
```

The Most Important Regex Symbols

Symbol	Meaning	Example	Simple meaning
^	Beginning of line	**^ERROR**	Lines starting with ERROR
$	End of line	**ERROR$**	Lines ending with ERROR
.	Any one character	**c.t**	cat, cut, cot
*****	Zero or more	**ab***	a, ab, abb...
[ ]	One character from a grou	**[abc]at**	aat, bat, cat
[0-9]	Any digit	**[0-9]**	0 through 9

^ Means: Beginning of the Line

Use ^ when the text must appear at the start.

Terminal

```
ERROR: disk full
WARNING: memory high
system ERROR
ERROR: permission denied
```

Terminal

```
$ grep "^ERROR" logfile.txt

ERROR: disk full
ERROR: permission denied
```

"^ERROR" = line starts with ERROR

$ Means: End of the Line

Use $ when the text must appear at the end.

Terminal

```
system error
network error
error occurred
disk error
ERROR
```

Terminal

```
$ grep "error$" logfile.txt

system error
network error
disk error
```

"error$" = line ends with error

. * [ ] — Easy Examples

These symbols make your search more flexible.

`.` → any one character: c.t can match cat, cut, cot

`*` → zero or more occurrences: ab* can match a, ab, abb…

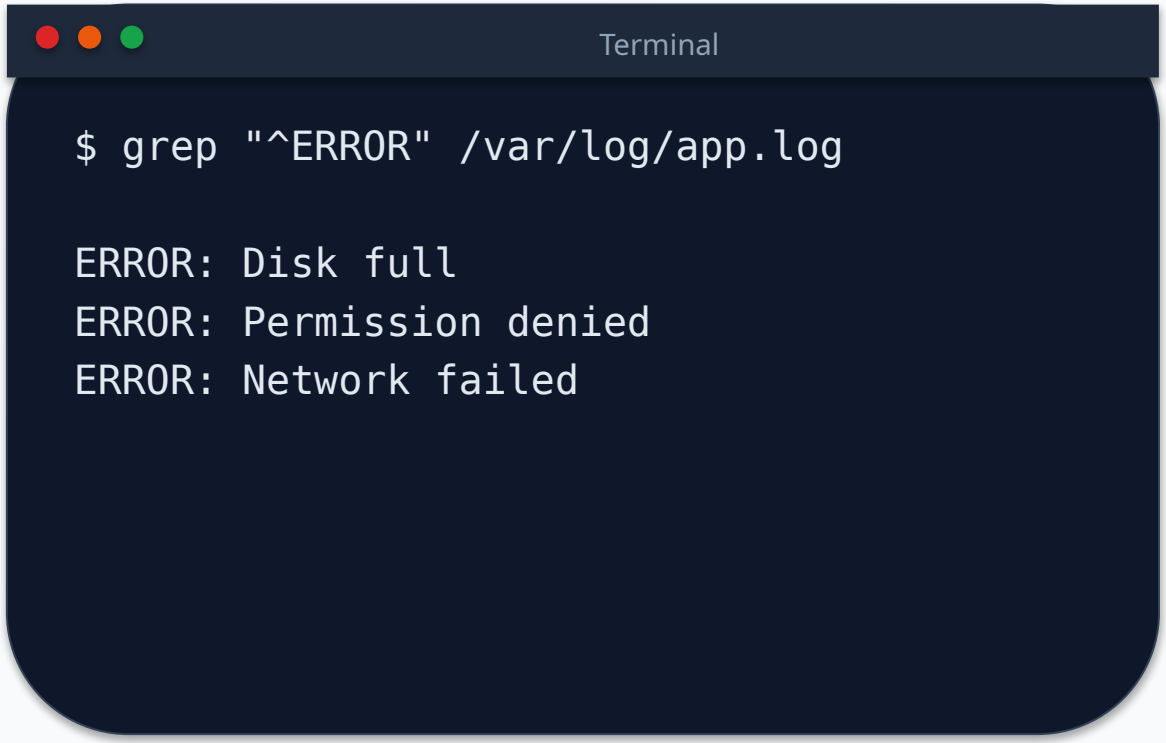
`[abc]` → one character from the group: [abc]at

`[0-9]` → any digit from 0 to 9



Real Linux Administrator Example

Finding errors in a server log.



Large log files are common in Linux administration.

Instead of reading every line, use a pattern.

`^ERROR` finds only lines that start with ERROR.

This quickly isolates problems that need attention.

Regex + grep = fast text filtering

Quick Memory Sheet

Keep these six patterns in mind.

`^` **Beginning**
`^ERROR`

`$` **Ending**
`ERROR$`

`.` **Any one character**
`c.t`

`*` **Zero or more**
`ab*`

`[ ]` **Choose one**
`[abc]at`

`[0-9]` **Any digit**
`[0-9]`

Regex = pattern → grep → matching text

RHEL ADMINISTRATION

Scheduling a Future User Job

at · atq · at -c · atrm

What deferred jobs are

The idea

Run commands once, at a specific future time. Example: a heavy maintenance task at midnight, or a safety job that resets a firewall in ten minutes unless you cancel it.

The tools

The **at** package is installed and enabled by default. It gives you the **atd** daemon plus the **at** and **atq** commands.

The queues

Any user can queue jobs. Queues run a to z and A to Z. Higher letters run with higher priority.

at TIMESPEC: queue a job

```
user@host:~$ at now +5min < myfile
warning: commands will be executed using /bin/sh
job 2 at Wed May 14 20:40:00 2025
```

now +5min run in five minutes

< myfile read the commands from a script file

job 2 the unique job number

Ctrl+D on an empty line ends manual typing

TIMESPEC: time, then date

TIMESPEC	Runs
now +5min	in 5 minutes
teatime tomorrow	16:00 tomorrow
noon +4 days	12:00, four days on
5pm august 3 2025	that exact date

Date only: time defaults to now. Time only: next time it matches.

```
user@host:~$ at 21:03 < myfile
job 3 at Wed May 14 21:03:00 2025
user@host:~$ at 21:00 < myfile
job 4 at Thu May 15 21:00:00 2025
21:00 had passed, so tomorrow
```

atq: list pending jobs

```
user@host:~$ atq
28 Mon May 19 05:13:00 2025 a user
29 Tue May 20 16:00:00 2025 h user
30 Wed May 21 12:00:00 2025 a user
```

28
unique job number

Mon May 19 05:13:00 2025
execution date and time

a
queue (default is a)

user
job owner

Inspect and remove a job

at -c JOBNUMBER

```
user@host:~$ at -c 28
(environment variables)
(commands that will run)
```

Shows the job's saved environment, copied from your shell when you queued it, then the commands.

atrm JOBNUMBER

```
user@host:~$ atrm 28
user@host:~$ atq
29 Tue May 20 16:00:00 2025 h user
```

Removes the job. Silent on success; atq confirms it is gone. Use it once the safety job is no longer needed.

Command cheat sheet

Command	What it does
at TIMESPEC < file	Queue a job from a script
atq or at -l	List your pending jobs
at -c JOBNUMBER	Show a job's environment and commands
atrm JOBNUMBER	Remove a scheduled job

RED HAT ENTERPRISE LINUX

Understanding & Creating Swap Space

A practical guide to virtual memory, swap files, swap partitions, and disk-based swap management in RHEL



10 SLIDES · SYSTEM ADMINISTRATION

What Is Swap Space?

A reserved area of disk that extends your system's usable memory

Definition

Swap space is disk storage used by the Linux kernel as an overflow for RAM.

When physical memory (RAM) fills up, inactive pages are moved to swap, freeing RAM for active processes.

It is slower than RAM but prevents applications from crashing due to out-of-memory errors.

RAM	SWAP
Fast	Slower
Volatile — cleared on reboot	Persistent on disk until removed

Where it lives

- A dedicated swap partition, or
- A swap file on an existing filesystem

Why Swap Space Is Needed

Four reasons swap remains a standard part of RHEL system design

Prevents OOM crashes

Gives the kernel room to move idle pages out of RAM instead of killing processes when memory runs low.

Supports hibernation

Suspend-to-disk writes the entire RAM image into swap so the system can resume later.

Extends effective memory

Lets a system handle memory spikes beyond installed RAM, useful on smaller VMs and servers.

Improves stability under load

The kernel can proactively swap out rarely used pages, keeping more RAM free for disk cache.

Two Ways to Configure Swap

RHEL supports swap on a dedicated partition or inside a regular file

Swap Partition

A dedicated disk partition (type Linux swap) used exclusively for swap.

PROS

Slightly faster; simple, predictable performance; standard on physical servers.

CONS

Fixed size; resizing requires repartitioning or LVM.

Swap File

A regular file on an existing filesystem, formatted and enabled as swap.

PROS

Easy to create, resize, or remove without touching partitions. Great for VMs and cloud instances.

CONS

Marginally slower on some filesystems (not on XFS/ext4 with proper setup).

Check Current Swap Status

Always confirm what's already configured before adding more swap

```
# View swap usage summary
$ free -h

# List active swap devices
$ swapon --show

# Detailed memory + swap info
$ cat /proc/swaps

# Check block devices & sizes
$ lsblk
```

What to look for

Total vs. Used

"free -h" shows how much swap exists and how much is in use right now.

Active devices

"swapon --show" lists every partition or file currently acting as swap, with type and priority.

Free disk space

Confirm there's enough free space on the target filesystem before creating a swap file.

Existing partitions

"lsblk" reveals unpartitioned or unused space that could become a swap partition.

Create a Swap File — Step by Step

The most flexible way to add swap on RHEL

1 Allocate the file

```
sudo fallocate -l 2G /swapfile
```

Reserves 2 GB. If fallocate isn't supported by the filesystem, use dd instead.

2 Secure the permissions

```
sudo chmod 600 /swapfile
```

Only root should be able to read or write the swap file.

3 Format as swap

```
sudo mkswap /swapfile
```

Writes a swap signature so the kernel recognizes the file.

4 Enable it

```
sudo swapon /swapfile
```

Activates the swap file immediately for the current session.

Create a Swap Partition

Use a dedicated partition when you need consistent, predictable performance

```
# 1. Create the partition (interactive)
$ sudo fdisk /dev/sdb
  Command (m for help): n     -> new partition
  Command (m for help): t     -> change type
  Hex code: 82                -> Linux swap
  Command (m for help): w     -> write & exit

# 2. Format the partition as swap
$ sudo mkswap /dev/sdb1

# 3. Activate it
$ sudo swapon /dev/sdb1
```

Good to know

Use a blank or unused disk/partition — this process is destructive to existing data.

On GPT disks, parted can be used instead of fdisk with type code 8200.

LVM logical volumes can also be formatted as swap the same way with mkswap.

Make Swap Permanent

swapon activates swap only until reboot — /etc/fstab makes it stick

```
# Back up fstab first
$ sudo cp /etc/fstab /etc/fstab.bak

# Append the swap entry
$ echo '/swapfile none swap sw 0 0' \
  | sudo tee -a /etc/fstab
```

For a partition, use its UUID instead of a path:

```
UUID=xxxxxxxx-xxxx  none  swap  sw  0 0
```

Find it with: blkid /dev/sdb1

fstab fields, explained

`Device`	Path or UUID of the swap file/partition
`none`	Swap has no mount point
`swap`	Filesystem type
`sw`	Mount option for swap
`0 0`	Dump and fsck order — always 0 for swap

Test it without rebooting:

```
sudo mount -a && sudo swapon -a
```

Managing & Tuning Swap

Common day-two operations once swap is up and running

Disable a swap file/partition

```
sudo swapoff /swapfile
```

Deactivates swap; do this before deleting it.

Remove a swap file

```
sudo rm /swapfile
```

Also delete its line from /etc/fstab.

Check swappiness

```
cat /proc/sys/vm/swappiness
```

0-100 scale; controls how aggressively the kernel swaps (default 60).

Adjust swappiness

```
sudo sysctl vm.swappiness=10
```

Lower values favor RAM; set permanently in /etc/sysctl.conf.

Best Practices & Recap

Sizing guidance and the key takeaways from this deck

Recommended Swap Sizing (RHEL guidance)

≤ 2 GB RAM	**2x RAM**
2-8 GB RAM	**Equal to RAM**
8-64 GB RAM	**0.5x RAM**
> 64 GB RAM	**4-8 GB (or per workload)**

Hibernation systems need swap ≥ total RAM, regardless of the table above.

Key Takeaways

- ✓ Swap extends RAM using disk — slower, but prevents OOM crashes
- ✓ Choose a swap file for flexibility, a partition for raw performance
- ✓ mkswap formats, swapon activates, /etc/fstab makes it permanent
- ✓ Verify with free -h and swapon --show after every change
- ✓ Size swap to workload — more isn't always better